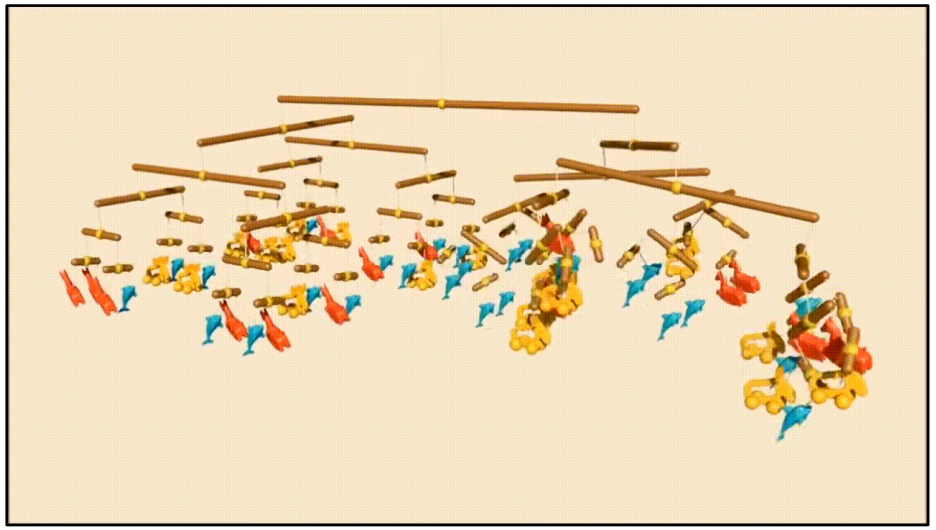
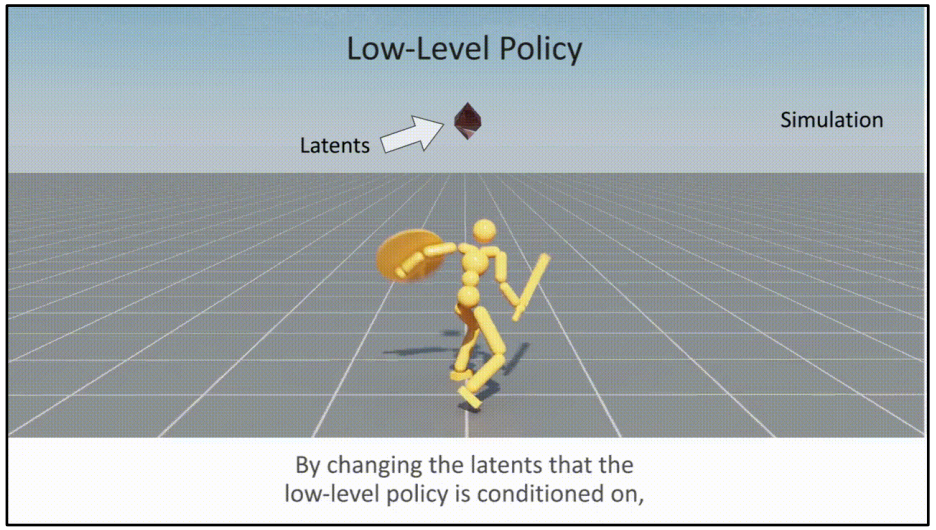
Physics Simulation for Computer Graphics

04 Rigid Body Collision

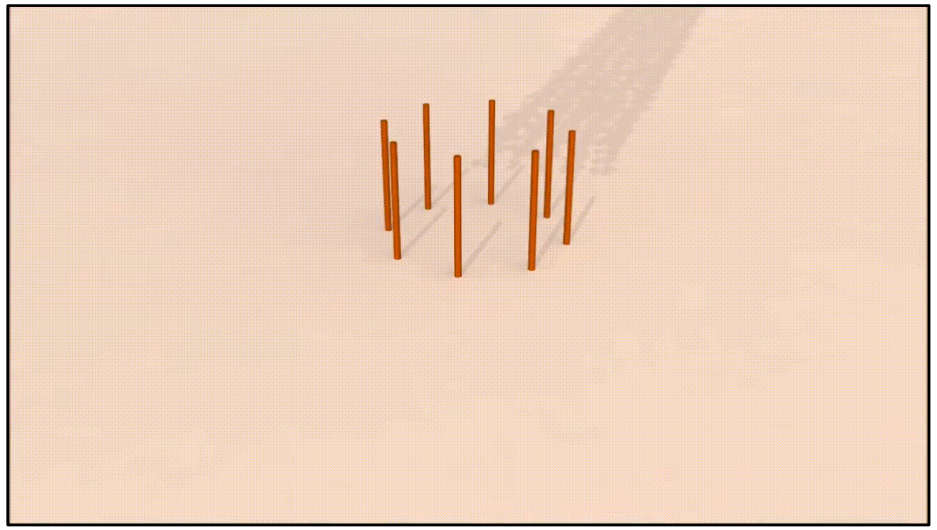
Rigid Body



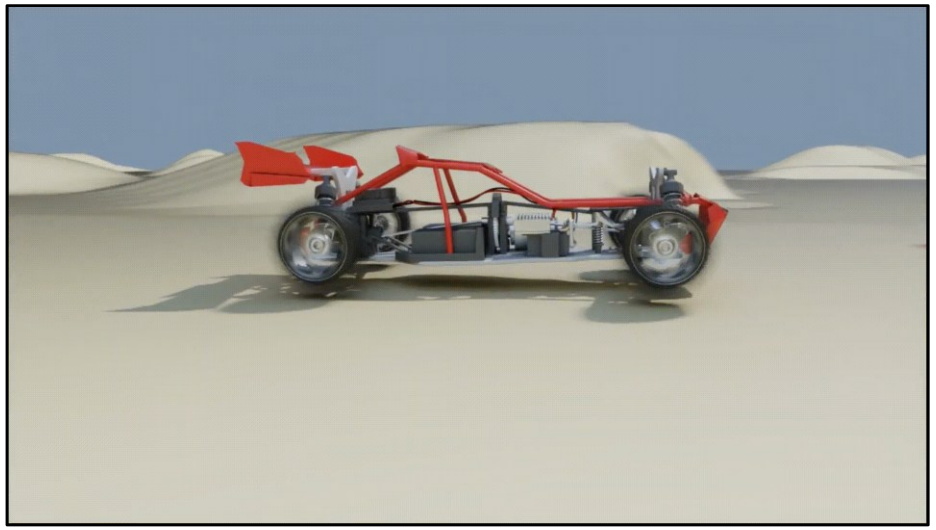
Deul et al. CAVW2014



Peng et al. SIGGRAPH2022



Deul et al. CAVW2014

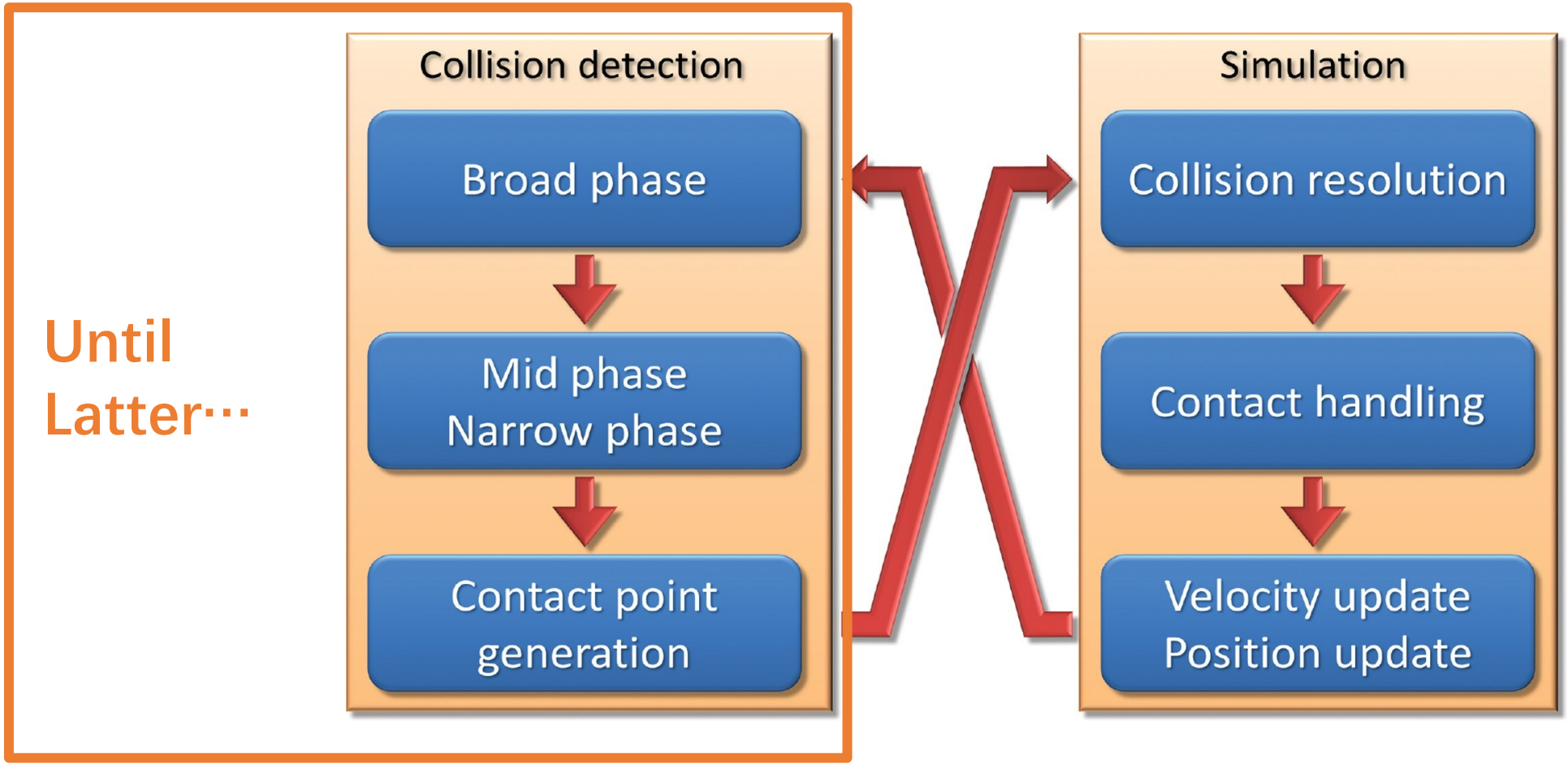


Muller et al. SCA2020

Rigid body Collision & Contact

Collision & Contact

= Collision Detection + (Collision Response + Contact Handling)



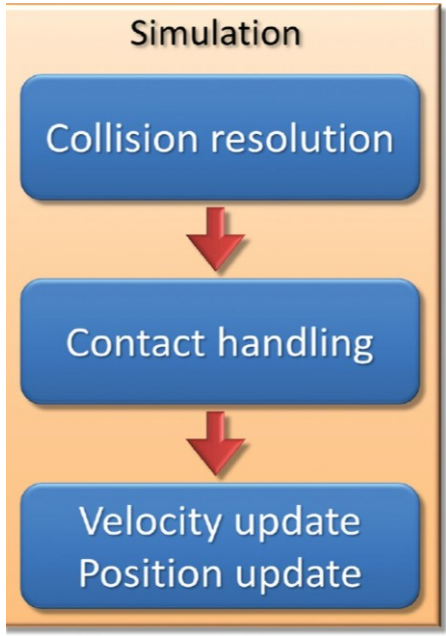
Rigid body Collision & Contact

Collision & Contact

= Collision Detection + (Collision Response + Contact Handling)

Simulate a Rigid-Body System:

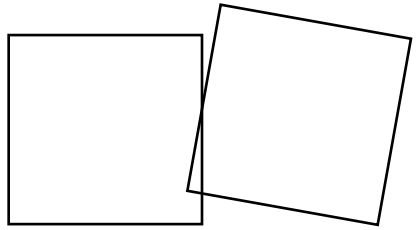
- Force-Based Methods: Particles & Rigid bodies
  - Penalty-based methods (Soft Collision and Contact)
  - Impulse-based methods (Hard Collision, Local Contact)
- Position-Based Method: Shape Matching (Collision+ Contact)
- Constraint-Based Methods:(Global Collision and Contact)
  - Solve a Complementarity Problem



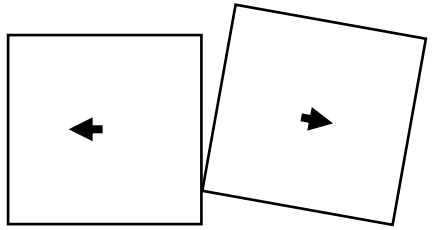
Until Latter…

Force-Based & Position-Based & Constraint-Based

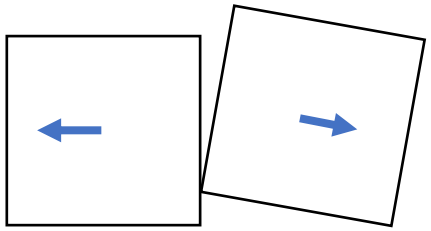
- Position-Based Update (PhysX)



penetration detection

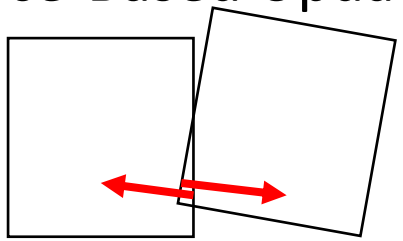


move objects so that they do not penetrate

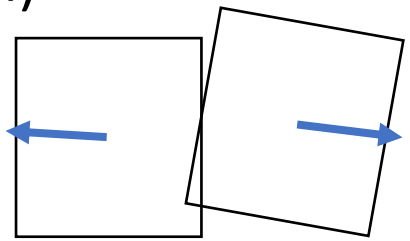


update velocities as position derivatives!

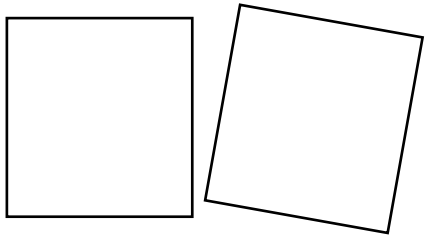
- Force-Based Update (Taichi)



penetration causes forces

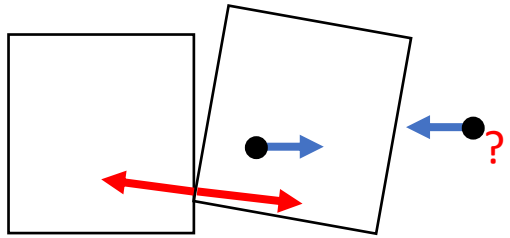


forces change velocities

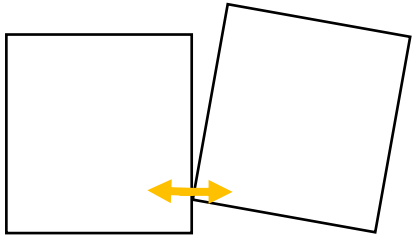


velocities change positions

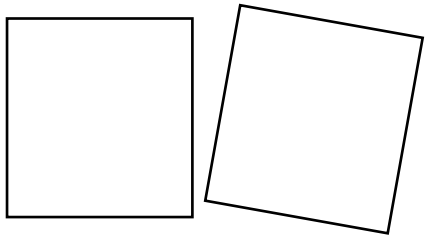
- Constraint-Based Update (MuJoCo)



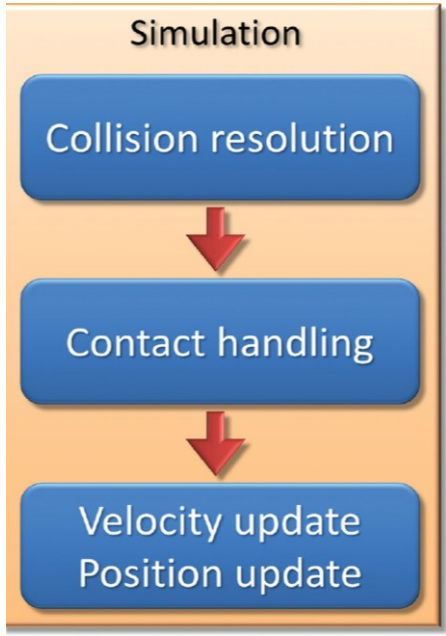
unknowns: (force, pos, vel)



equations: physics, constraints



solve globally and update all



Force-Based & Position-Based & Constraint-Based

Best Practices in Choosing a Method:

- Games → Position-based
- High-Fidelity Simulation → Impulse + Constraint
- Robotics → Constraint-based

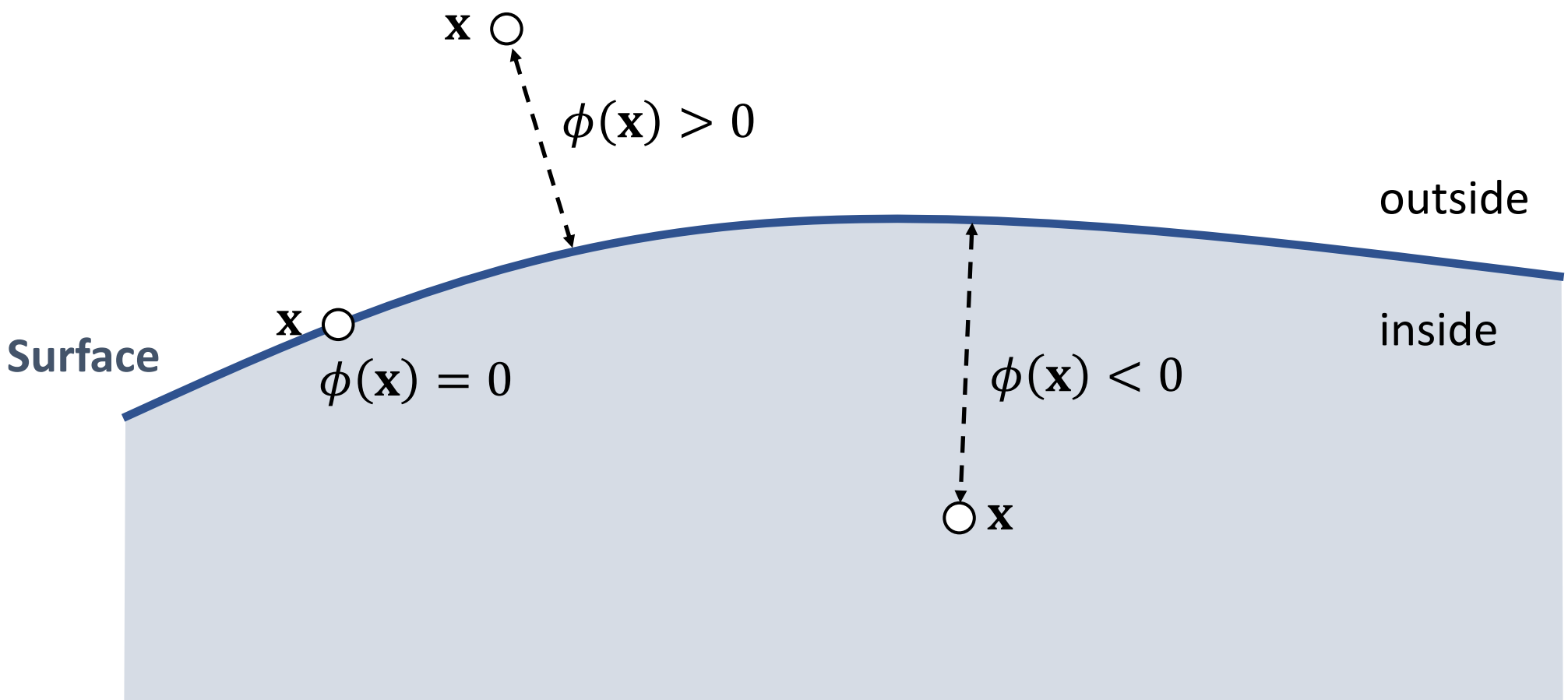
Method	Accuracy	Stability	Performance
Force-Based	Moderate	Needs tuning	Medium
Impulse-Based	High (for collisions)	Unstable for stacking	Medium
Position-Based	Low (not force-based)	**Very Stable**	**High**
Constraint-Based (LCP)	**High** (Global Solve)	Stable	Expensive

Force-Based: Particle Collision

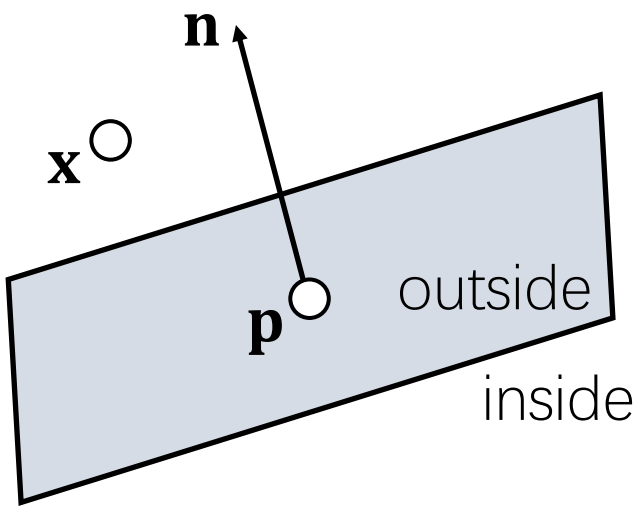
Detection and Response

Signed Distance Function

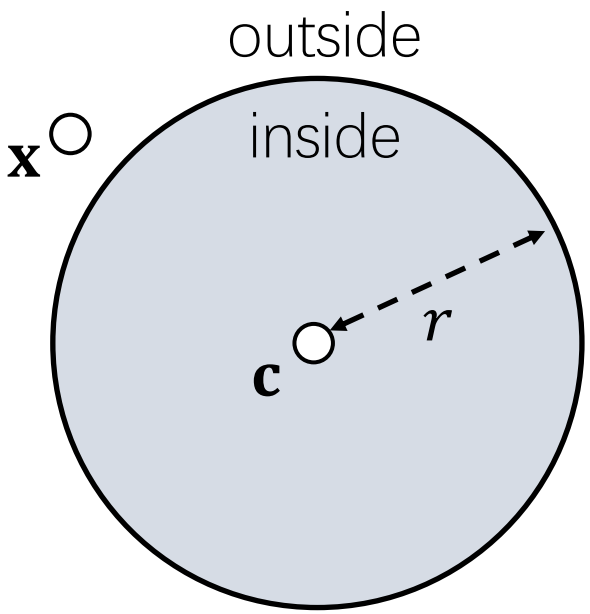
A <u>signed</u> distance function $\phi(\mathbf{x})$ defines the distance from $\mathbf{x}$ to a surface with a sign. The sign indicates on which side $\mathbf{x}$ is located.



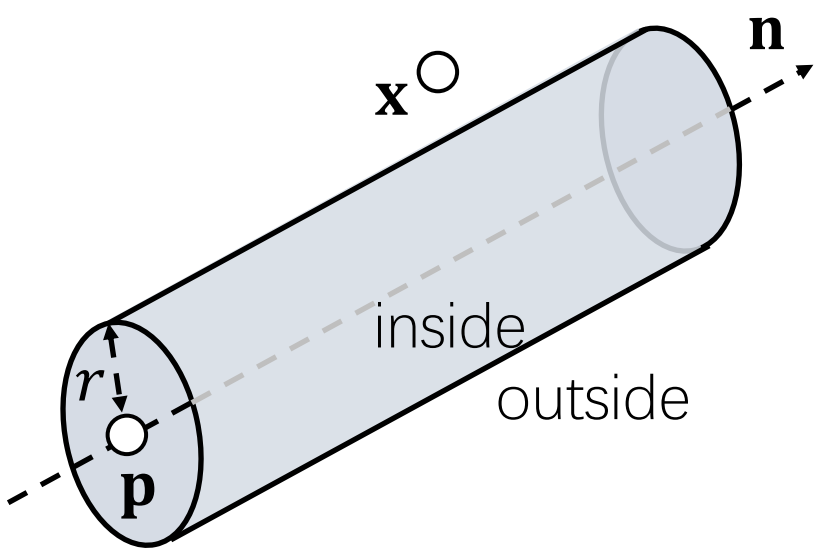
Signed Distance Function Examples



$$\phi(\mathbf{x}) = (\mathbf{x} - \mathbf{p}) \cdot \mathbf{n}$$

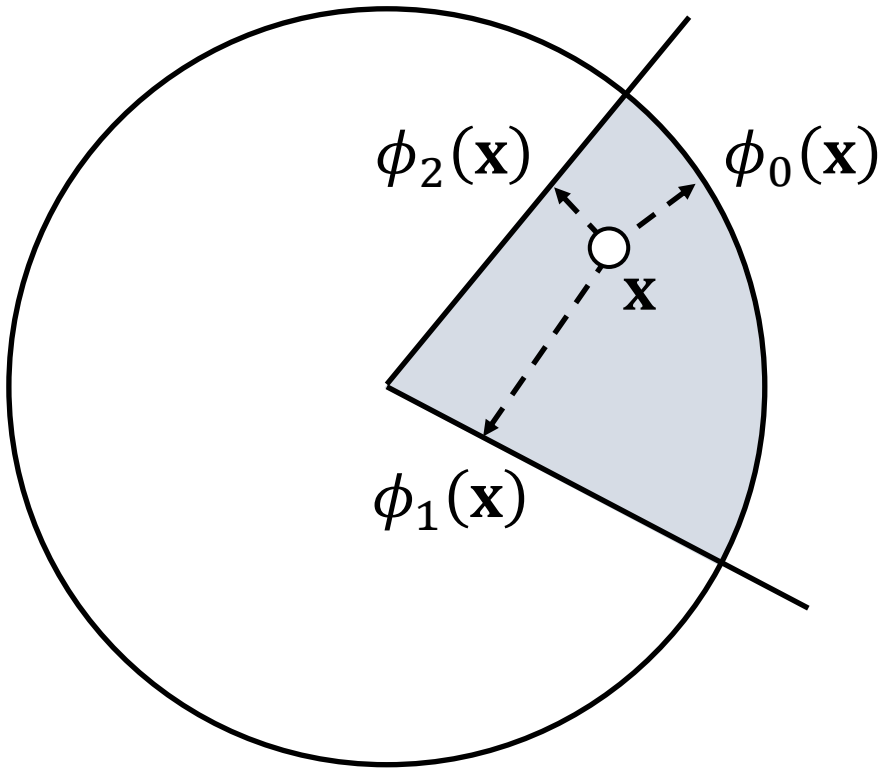


$$\phi(\mathbf{x}) = \|\mathbf{x} - \mathbf{c}\| - r$$



$$\phi(\mathbf{x}) = \sqrt{\|\mathbf{x} - \mathbf{p}\|^2 - \left((\mathbf{x} - \mathbf{p}) \cdot \mathbf{n}\right)^2} - r$$

Intersection of Signed Distance Functions



If $\phi_0(\mathbf{x}) < 0$ and $\phi_1(\mathbf{x}) < 0$ and $\phi_2(\mathbf{x}) < 0$

// inside

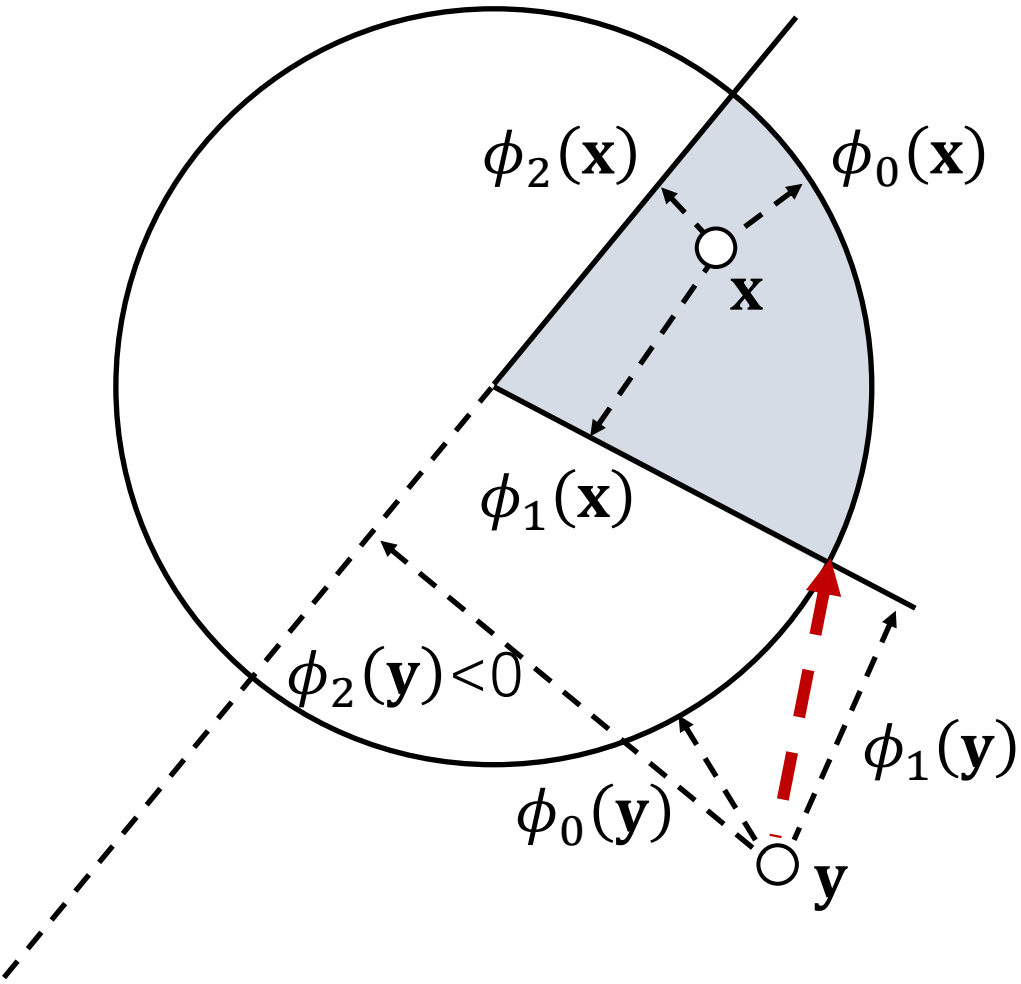
$\phi(\mathbf{x}) = \max\big(\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \phi_2(\mathbf{x})\big)$

Else

//outside

$\phi(\mathbf{x}) = ?$

Intersection of Signed Distance Functions



If $\phi_0(\mathbf{x}) < 0$ and $\phi_1(\mathbf{x}) < 0$ and $\phi_2(\mathbf{x}) < 0$

// inside

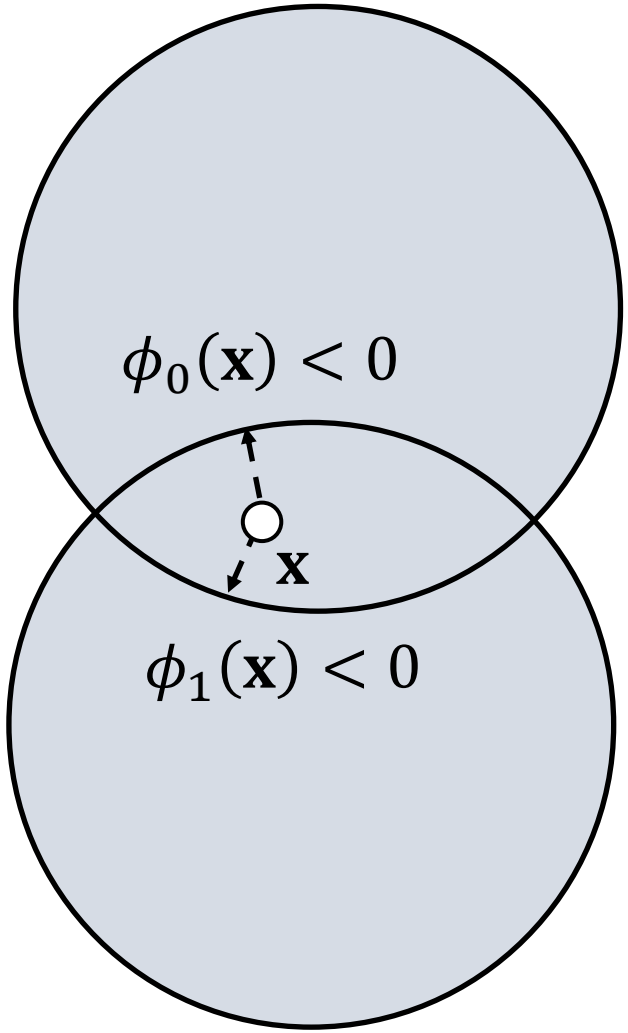
$$\phi(\mathbf{x}) = \max\big(\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \phi_2(\mathbf{x})\big)$$

Else

//outside

$$\phi(\mathbf{x}) \geq \max\big(\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \phi_2(\mathbf{x})\big)$$

Union of Signed Distance Functions



If $\phi_0(\mathbf{x}) < 0$ or $\phi_1(\mathbf{x}) < 0$

then inside

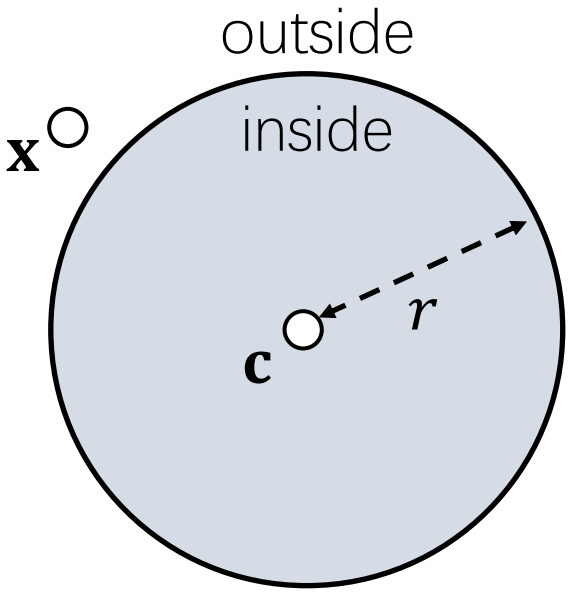
$$\phi(\mathbf{x}) \boxed{\leq} \min\big(\phi_0(\mathbf{x}), \phi_1(\mathbf{x})\big)$$

Else outside ≈ near outer boundary

$$\phi(\mathbf{x}) = \min\big(\phi_0(\mathbf{x}), \phi_1(\mathbf{x})\big)$$

Intuitively, we can consider collision detection with the union of two objects as collision detection with two separate objects.

Representations of SDF



$$\phi(\mathbf{x}) = \|\mathbf{x} - \mathbf{c}\| - r$$

Eikonal equations:

$$\| \nabla f \| = 1$$

Zhao H. A fast sweeping method for eikonal equations[J]. Mathematics of computation, 2005

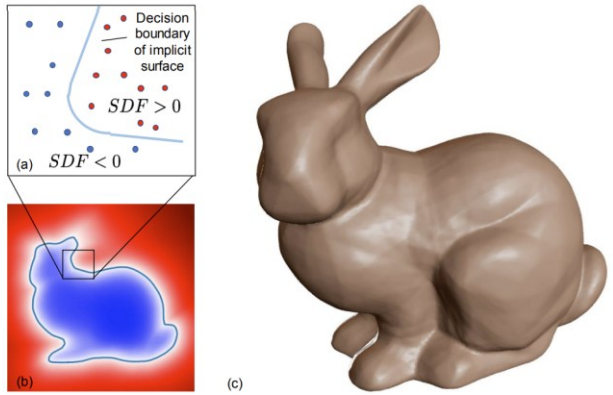


Figure 2: Our DeepSDF representation applied to the Stanford Bunny: (a) depiction of the underlying implicit surface $SDF = 0$ trained on sampled points inside $SDF < 0$ and outside $SDF > 0$ the surface, (b) 2D cross-section of the signed distance field, (c) rendered 3D surface recovered from $SDF = 0$. Note that (b) and (c) are recovered via DeepSDF.

Park J J, Florence P, Straub J, et al. Deepsdf: Learning continuous signed distance functions for shape representation[C]. CVPR 2019

Quadratic Penalty Method

A penalty method applies a penalty force in the next update.
When the penalty potential is quadratic, the force is linear.

Quadratic penalty potential

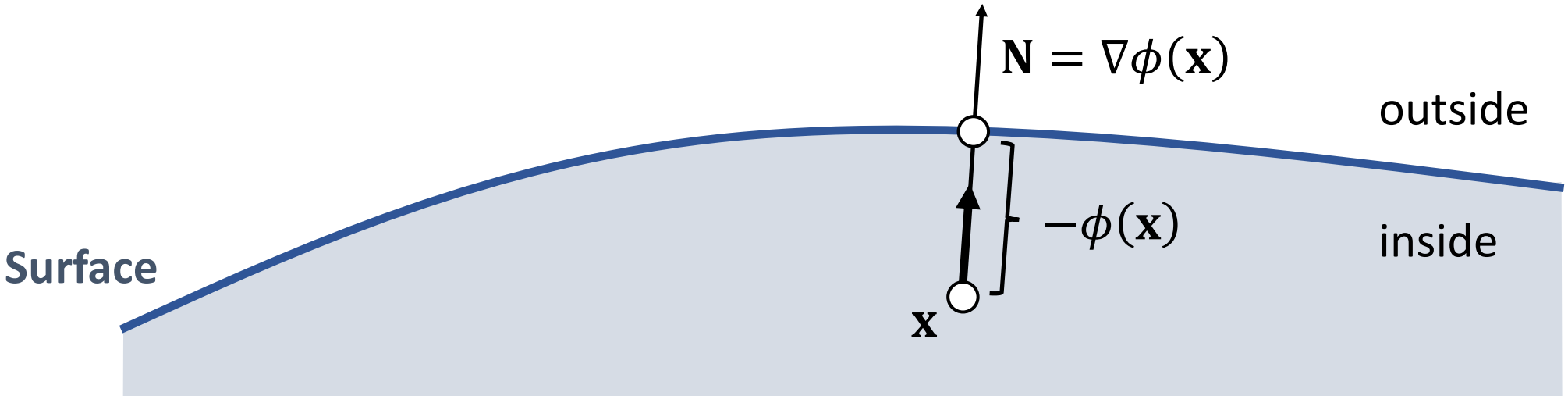
$$\mathrm{E} \longleftarrow -\frac{1}{2} k \phi^2(\mathbf{x})$$

Is $\phi(\mathbf{x}) < 0$? —yes→ Apply a force in the next update:

$$\mathbf{f} \longleftarrow -k\ \phi(\mathbf{x})\ \mathbf{N}$$

penalty strength

no ↓ Done



Quadratic Penalty Method

A penalty method applies a penalty force in the next update.
When the penalty potential is quadratic, the force is linear.

Quadratic penalty potential

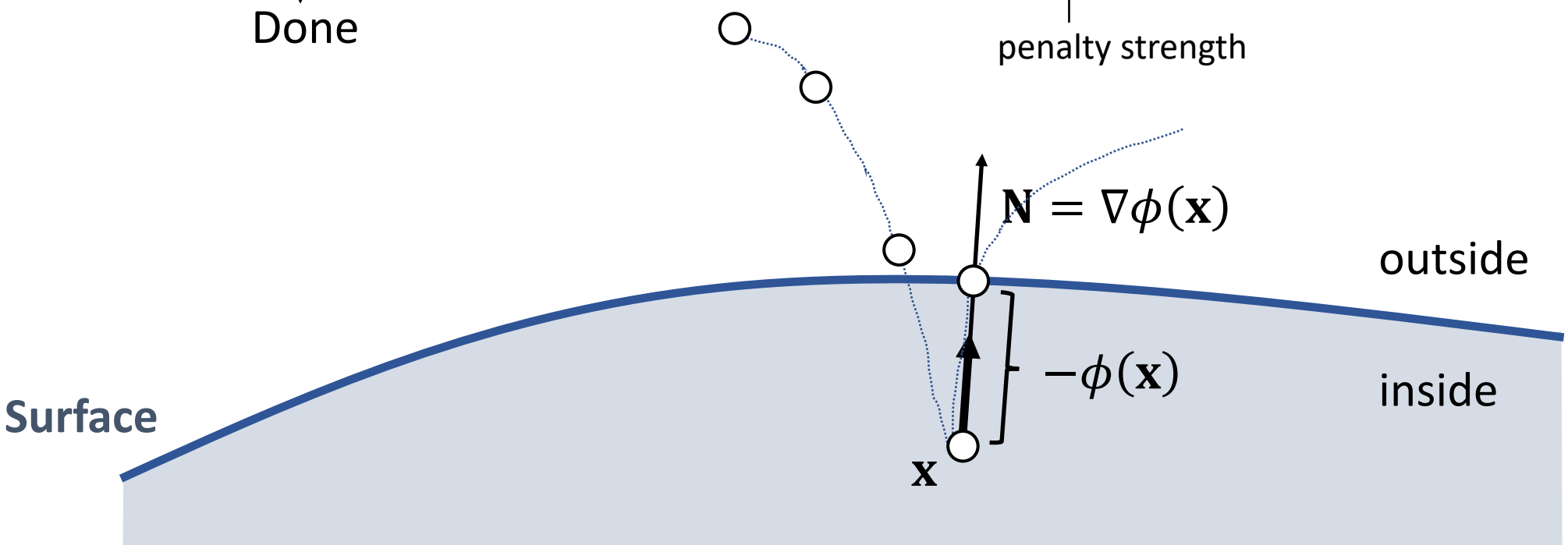
$$\mathrm{E} \longleftarrow -\frac{1}{2} k \phi^2(\mathbf{x})$$

Is $\phi(\mathbf{x}) < 0$? yes → Apply a force in the next update:

$$\mathbf{f} \longleftarrow -k\ \phi(\mathbf{x})\ \mathbf{N}$$

penalty strength

no ↓ Done



Quadratic Penalty Method with a Buffer

A buffer helps lessen the penetration issue.
But it cannot strictly prevent penetration,
no matter how large k is.

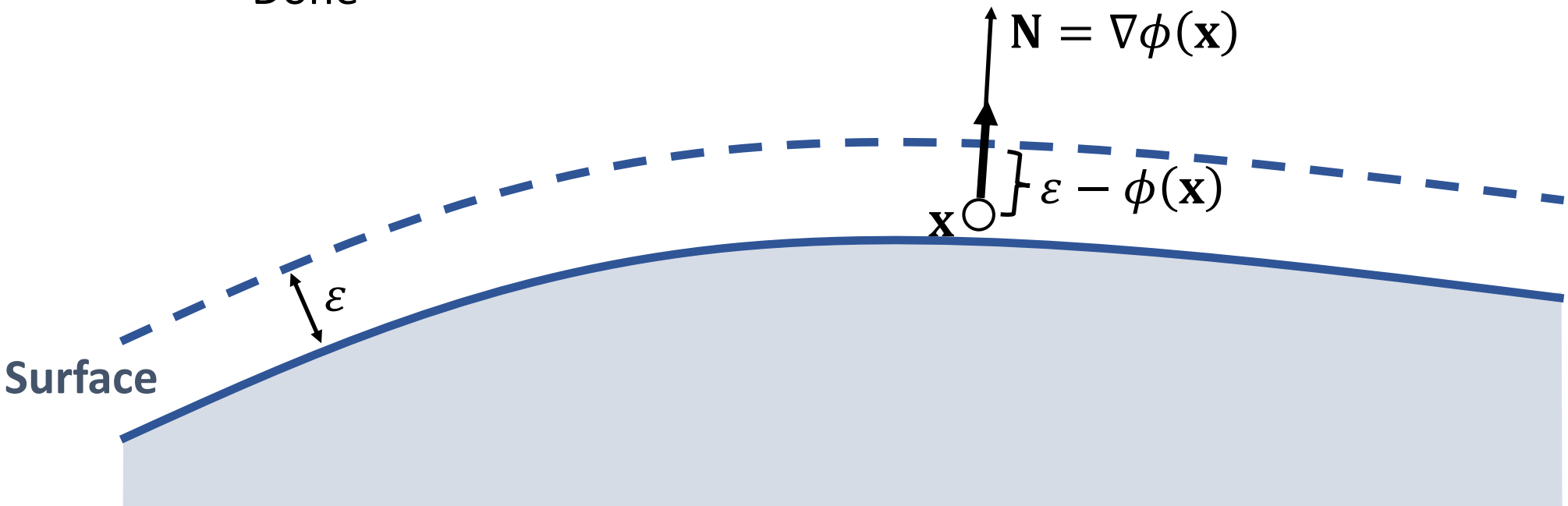
Quadratic penalty potential

$$\mathrm{E} \longleftarrow -\frac{1}{2}k[\varepsilon - \phi(\mathbf{x})]^2$$

Is $\phi(\mathbf{x}) < \varepsilon$? yes → Apply a force in the next update:

$$\mathbf{f} \longleftarrow k\big(\varepsilon - \phi(\mathbf{x})\big)\mathbf{N}$$

no ↓ Done



Quadratic Penalty Method with a Buffer

A buffer helps lessen the penetration issue.
But it cannot strictly prevent penetration,
no matter how large k is.

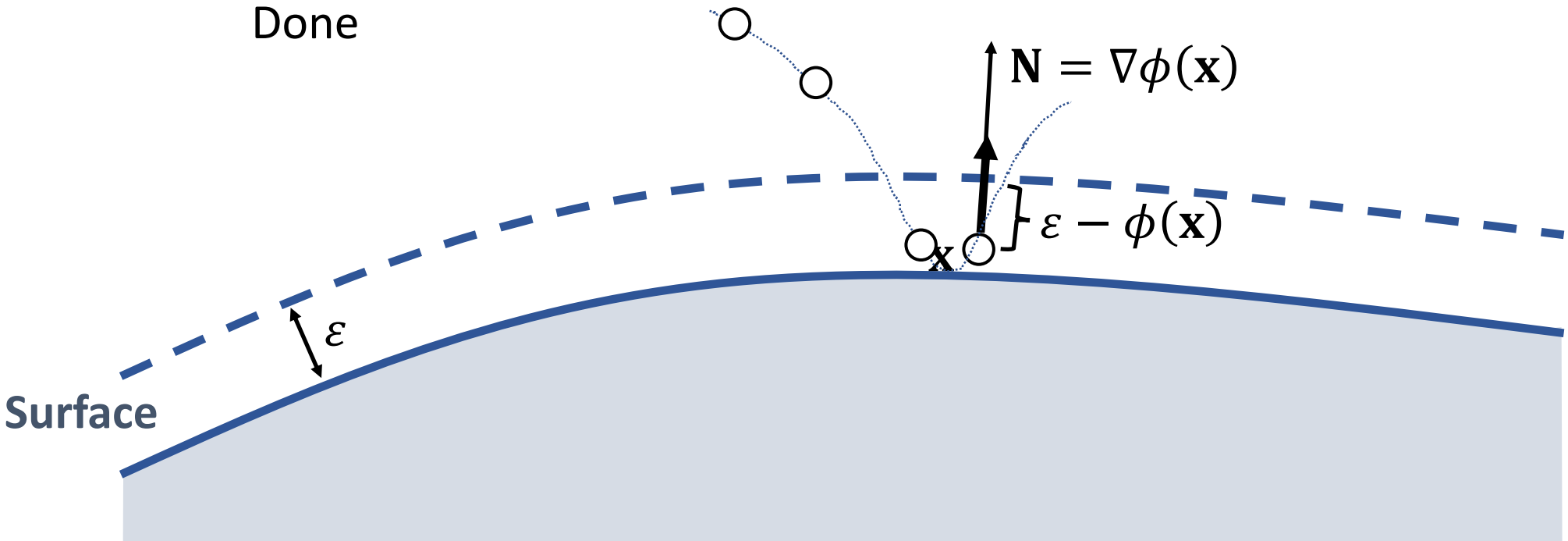
Quadratic penalty potential

$$\mathrm{E} \longleftarrow -\frac{1}{2}k[\varepsilon - \phi(\mathbf{x})]^2$$

Is $\phi(\mathbf{x}) < \varepsilon$? yes → Apply a force in the next update:

$$\mathbf{f} \longleftarrow k\big(\varepsilon - \phi(\mathbf{x})\big)\mathbf{N}$$

no ↓ Done



Log-Barrier Penalty Method

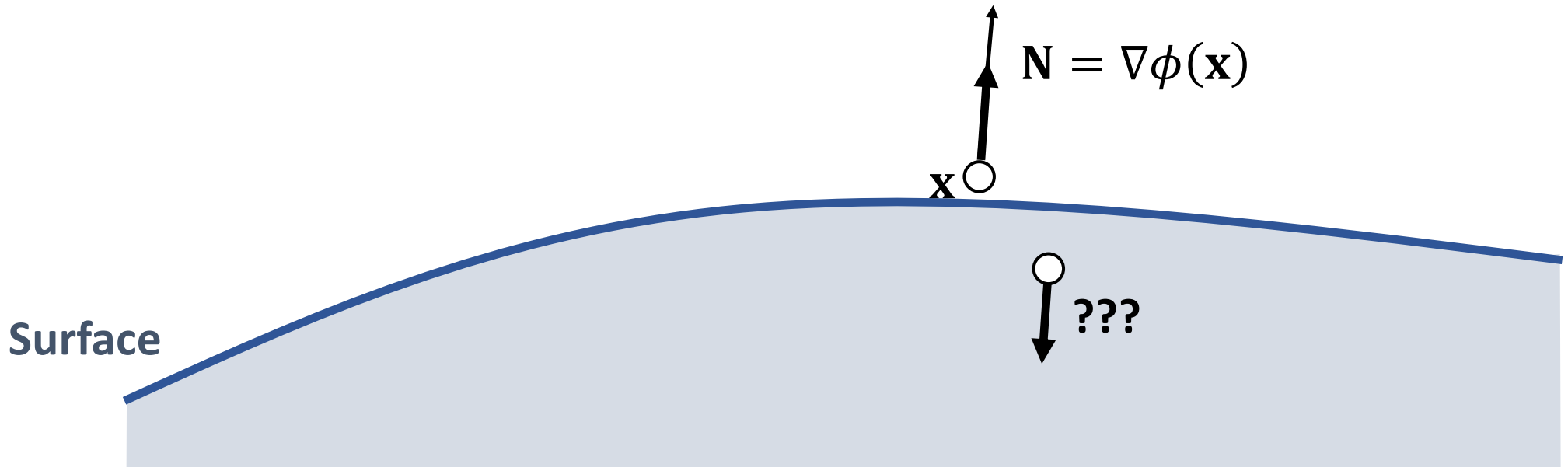
A log-barrier penalty potential ensures that the force can be large enough. But it assumes $\phi(\mathbf{x}) < 0$ will never happen!!! To achieve that, it needs to adjust Δt.

Always apply the penalty force as:

$$\mathbf{f} \longleftarrow \underbrace{\rho}_{\text{barrier strength}} \frac{1}{\phi(\mathbf{x})} \mathbf{N}$$

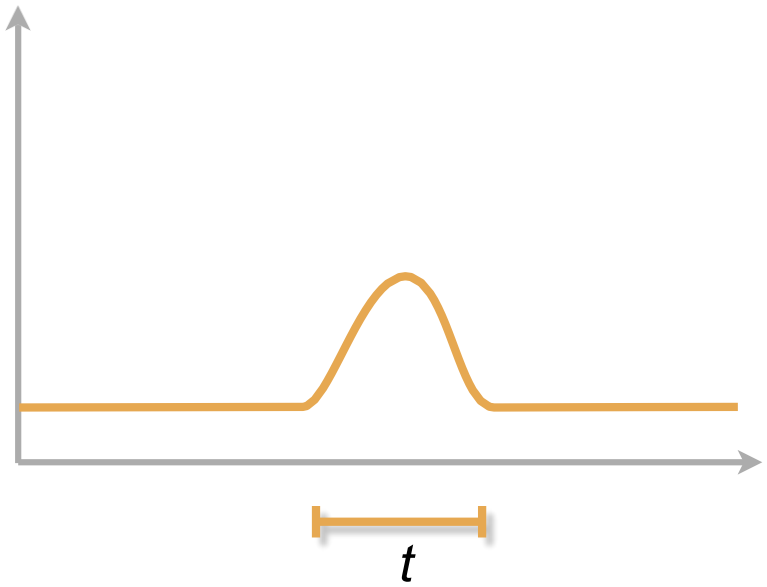
Log-Barrier penalty potential

$$\mathrm{E} \longleftarrow \rho \ln(\phi(\mathbf{x}))$$

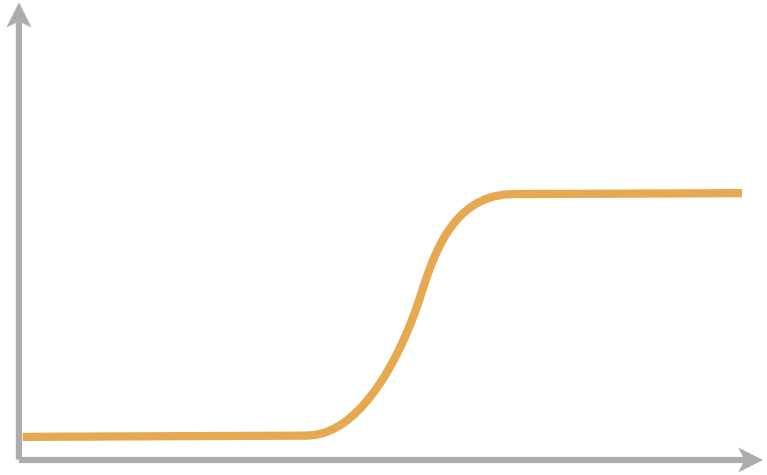


Soft Collision

- Force

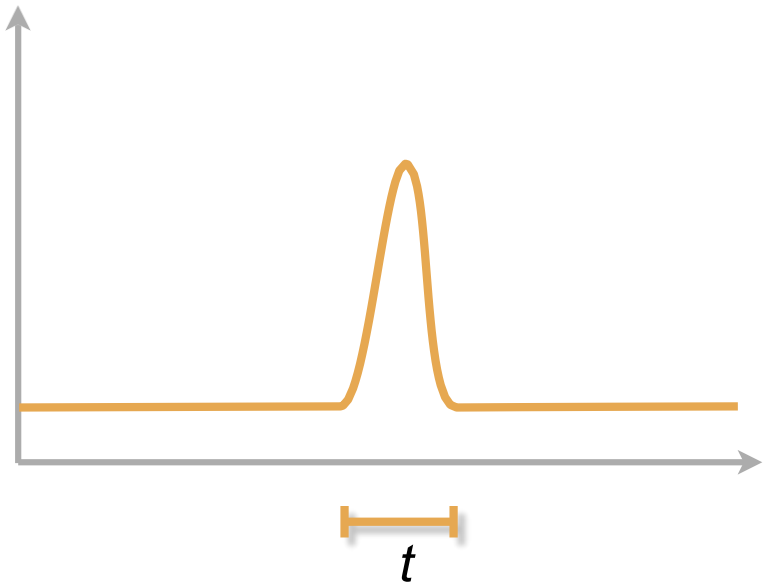


- Velocity

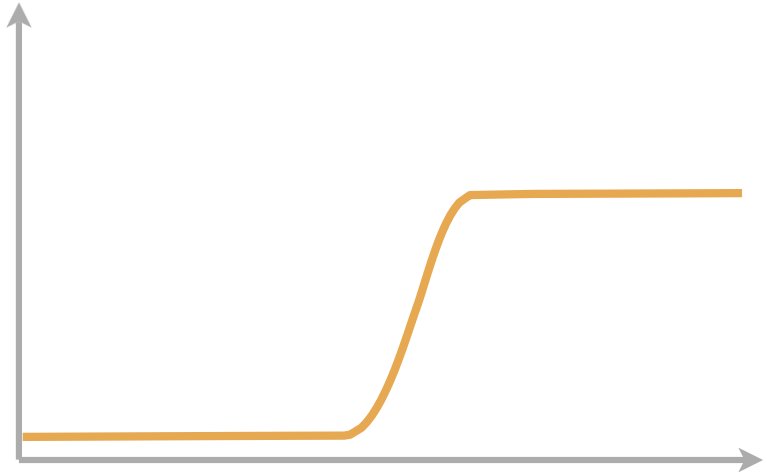


Harder Collision

- Force

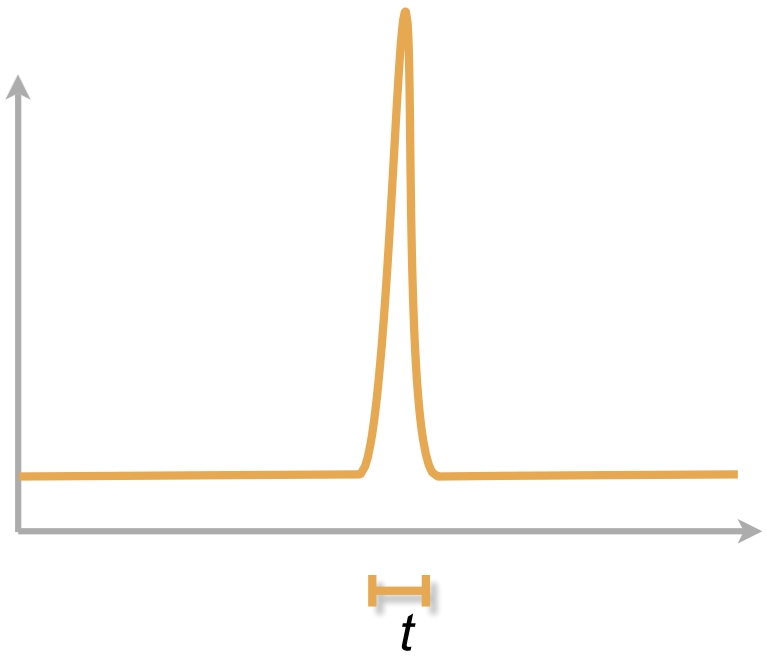


- Velocity

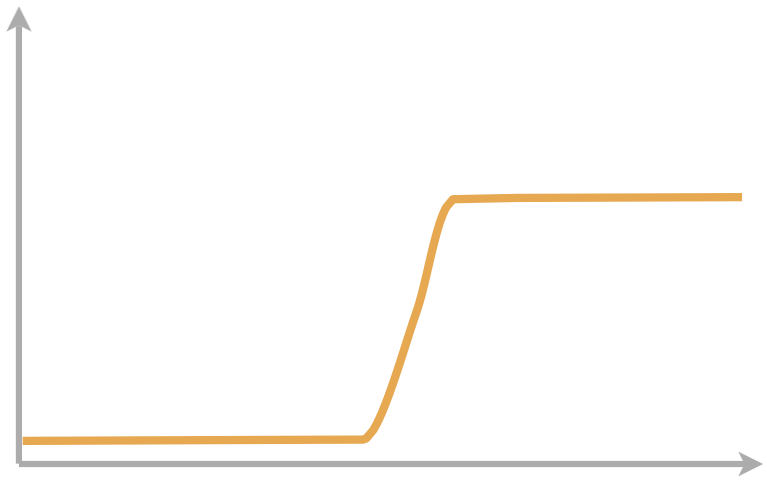


Very Hard Collision

- Force

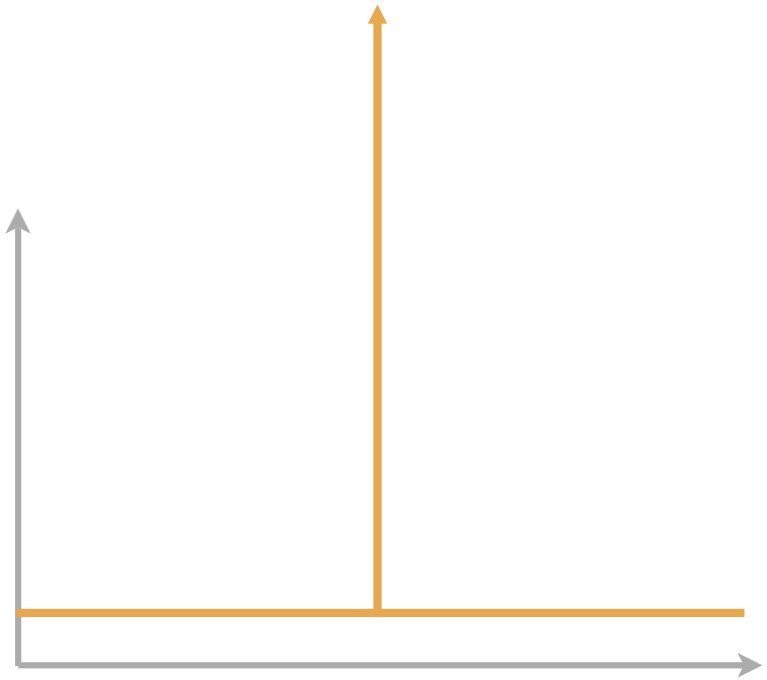


- Velocity



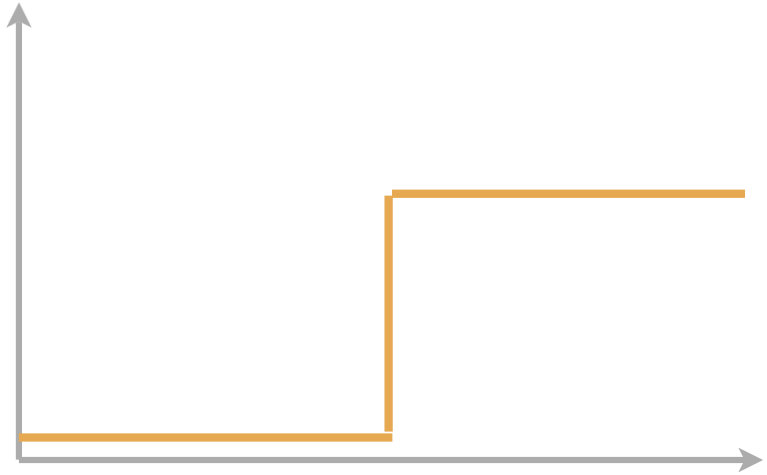
Impulse Method

- Impulsive force



t=0 , infinite force

- Velocity



A Short Summary of Penalty Methods

- The use of step size adjustment is a must.
  - To avoid overshooting.
  - To avoid penetration in log-barrier methods.
- Log-barrier method can be limited within a buffer as well.
  - Li et al. 2020. *Incremental Potential Contact: Intersection- and Inversion-free Large Deformation Dynamics*. TOG.
  - Wu et al. 2020. *A Safe and Fast Repulsion Method for GPU-based Cloth Self Collisions*. TOG.
- Frictional contacts are difficult to handle.

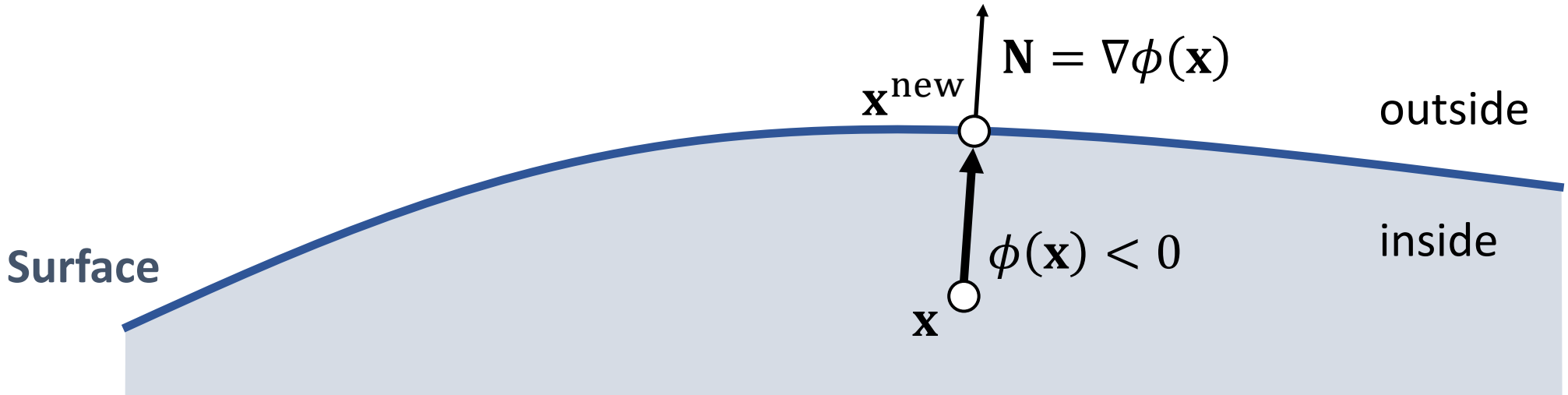
Impulse Method

An impulse method assumes that collision changes the position and the velocity all of sudden.

Is $\phi(\mathbf{x}) < 0$? —yes→ collision: $\mathbf{x}^{\text{new}} \longleftarrow \mathbf{x} + |\phi(\mathbf{x})|\mathbf{N} = \mathbf{x} - \phi(\mathbf{x})\nabla\phi(\mathbf{x})$

no ↓ Done

What about velocity and friction?

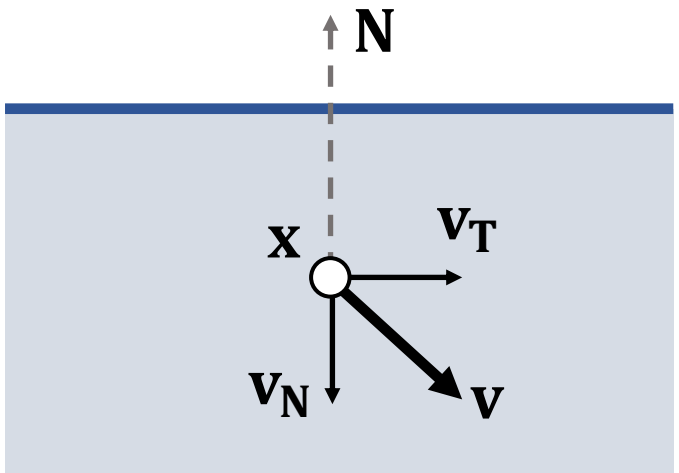


Impulse Method

Changing the position is not enough, we must change the velocity as well.

Is $\mathbf{v} \cdot \mathbf{N} < 0$? yes → $\begin{cases} \mathbf{v_N} \longleftarrow (\mathbf{v} \cdot \mathbf{N})\mathbf{N} \\ \mathbf{v_T} \longleftarrow \mathbf{v} - \mathbf{v_N} \end{cases}$ ⇒ $\begin{cases} \mathbf{v_N^{new}} \longleftarrow -\mu_\mathbf{N}\mathbf{v_N} \\ \mathbf{v_T^{new}} \longleftarrow a\,\mathbf{v_T} \end{cases}$ ⇒ $\mathbf{v}^{\text{new}} \longleftarrow \mathbf{v_N^{new}} + \mathbf{v_T^{new}}$

no ↓ Done



a should not violating Coulomb's law

$$\|\mathbf{v_T^{new}} - \mathbf{v_T}\| \le \mu_\mathbf{T}\|\mathbf{v_N^{new}} - \mathbf{v_N}\|$$

$$(1-a)\|\mathbf{v_T}\| \le \mu_\mathbf{T}(1+\mu_\mathbf{N})\|\mathbf{v_N}\|$$

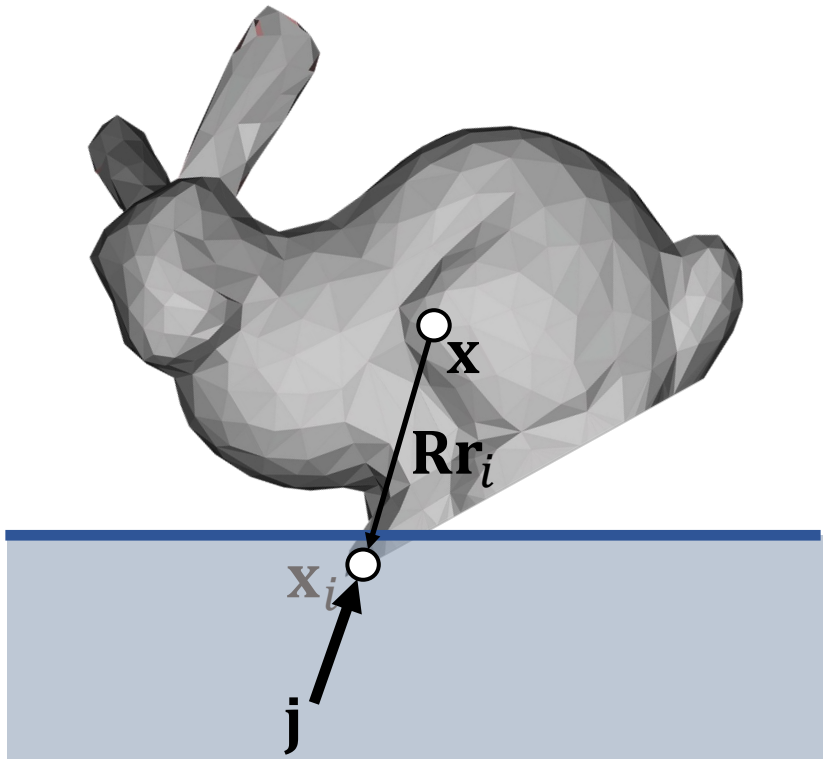
Therefore,

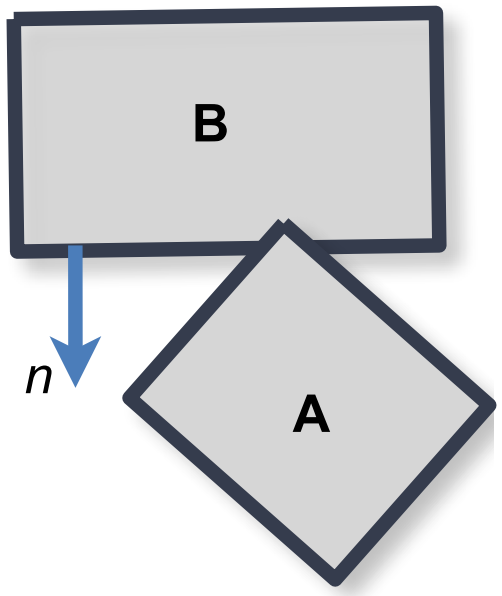
$$a \longleftarrow \max(\underbrace{1 - \mu_\mathbf{T}(1+\mu_\mathbf{N})\|\mathbf{v_N}\|/\,\|\mathbf{v_T}\|}_{\text{dynamic friction}}, \underbrace{0}_{\text{static friction}})$$

Force-Based: Rigid Body Collision

Impulse Method

Rigid Body Collision Response by Impulse





Update rigid body velocity:

$$\begin{cases} \mathbf{v}^{\text{new}} \leftarrow \mathbf{v} + \frac{1}{M}\mathbf{j} \\ \boldsymbol{\omega}^{\text{new}} \leftarrow \boldsymbol{\omega} + \mathbf{I}^{-1}\boxed{(\mathbf{R}\mathbf{r}_i \times \mathbf{j})} \end{cases}$$

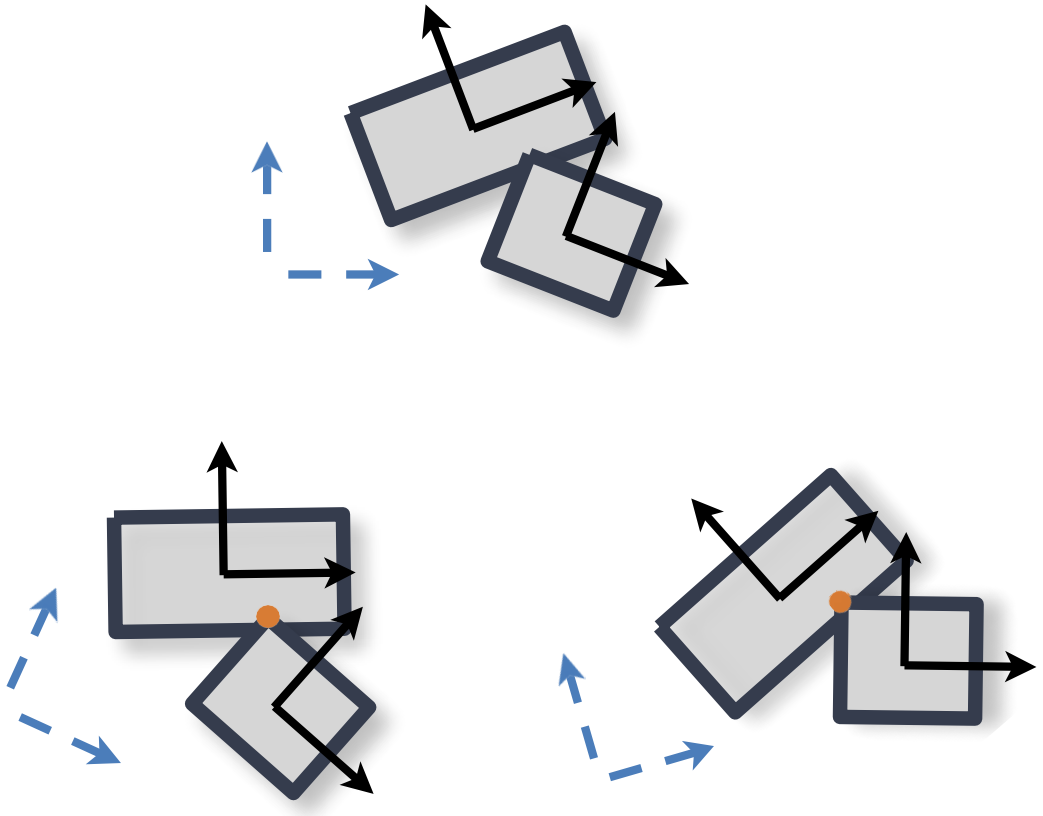
torque induced by **j**

$$\mathbf{v}'_a = \mathbf{v}_a + J\mathbf{n}/M_a$$
$$\mathbf{v}'_b = \mathbf{v}_b - J\mathbf{n}/M_b$$

$$\mathbf{L}'_a = \mathbf{L}_a + (\mathbf{x}_a \times J\mathbf{n})$$
$$\mathbf{L}'_b = \mathbf{L}_b - (\mathbf{x}_b \times J\mathbf{n})$$

Frictionless Collision Between Two Boxes

- Collision Detection (very simple approach):
  - Test in the Local Space of the target body (undo translation & rotation)
  - Check corner points
  - Check points on the edges (or points on the surface)
  - Return contact points (0 or 1 or multiple)



Frictionless Collision Between Two Boxes

碰撞检测

- 我们不要求你实现一个碰撞检测算法，感兴趣的同学可以阅读参考资料了解算法和具体实现。
- 你可以使用 The Flexible Collision Library (fcl) 实现你自己的碰撞检测，在 Lab0 中你应该已经安装了 fcl。你可以阅读 fcl 文档了解相关的类定义，或者阅读 fcl 提供的示例代码。我们提供一种可能的实现以供参考：

```
void RigidBodySystem::collisionDetectBoxBox_fcl(int id1, int id2) {
    auto const &        b0 = _bodies[id1];
    auto const &        b1 = _bodies[id2];
    using CollisionGeometryPtr_t = std::shared_ptr<fcl::CollisionGeometry<float>>;
    // Eigen::Vector3f RigidBody::dim, size of a box
    CollisionGeometryPtr_t box_geometry_A(new fcl::Box<float>(b0.dim[0], b0.dim[1], b0.dim[2]));
    CollisionGeometryPtr_t box_geometry_B(new fcl::Box<float>(b1.dim[0], b1.dim[1], b1.dim[2]));
    // Eigen::Vector3f RigidBody::x, position of a box; Eigen::Quaternionf RigidBody::q, rotation of a box
    fcl::CollisionObject<float> box_A(box_geometry_A, fcl::Transform3f(Eigen::Translation3f(b0.x)*b0.q));
    fcl::CollisionObject<float> box_B(box_geometry_B, fcl::Transform3f(Eigen::Translation3f(b1.x)*b1.q));
    // Compute collision - at most 8 contacts and return contact information.
    fcl::CollisionRequest<float> collisionRequest(8, true);
    fcl::CollisionResult<float> collisionResult;
    fcl::collide(&box_A, &box_B, collisionRequest, collisionResult);
    if(! collisionResult.isCollision()) return;
    std::vector<fcl::Contact<float>> contacts;
    collisionResult.getContacts(contacts);
    for(auto const & contact : contacts) { // You can decide whether define your own Contact
        _contacts.emplace_back(Contact(id1, id2, contact.pos, contact.normal, contact.penetration_depth));
    }
}
```

FCL features:

- C++
- Either linux or win32
- Supported shapes:
  - box
  - sphere
  - ellipsoid
  - capsule
  - cone
  - cylinder
  - convex
  - half-space
  - plane
  - mesh
  - octree

Frictionless Collision Between Two Boxes

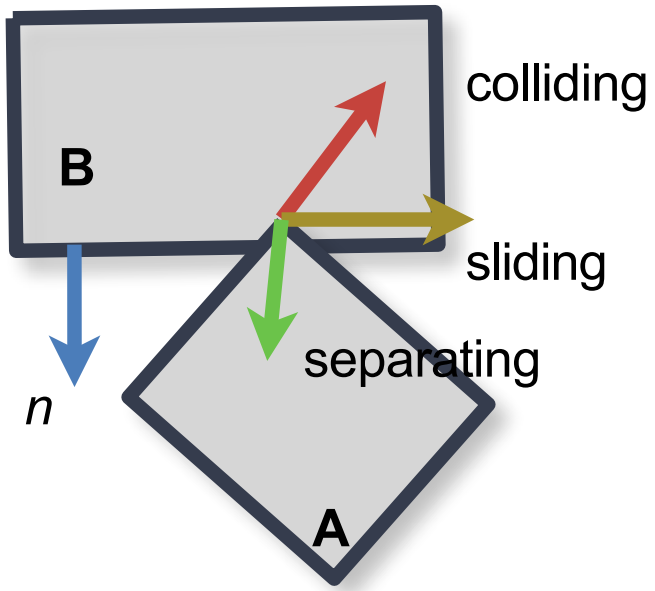
- Classifying Contacts:
  - Velocity of $\mathbf{x}_i$ on rigid body:

$$\mathbf{x}_i = \mathbf{x} + \mathbf{R}\mathbf{r}_i \qquad \mathbf{v}_i = \mathbf{v}_{cm} + \boldsymbol{\omega} \times \mathbf{R}\mathbf{r}_i$$

  - Retrieve collision normal: $\boldsymbol{n}$
  - Compute relative velocity:

$$v_{rel} = \boldsymbol{n} \cdot (\mathbf{v}_A - \mathbf{v}_B)$$

  - 3 Cases:
    - $v_{\text{rel}} < 0$, Colliding contact
    - $v_{\text{rel}} > 0$, Separating (easy!)
    - $v_{\text{rel}} = 0$, Sliding contact

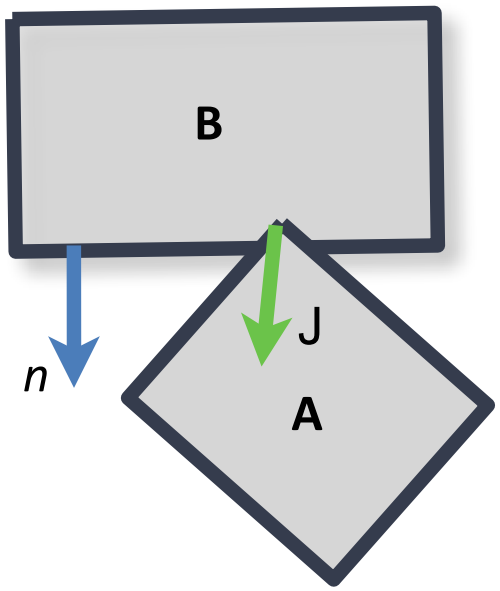


- Data returned by the method:

```
struct CollisionInfo{
        bool    isValid;
        Vec3    collisionPointWorld;
        Vec3    normalWorld;
        float   depth;
}
```

Frictionless Collision Between Two Boxes

- Collision Response
  - Compute instantaneous effect of material deformation (J)
  - Separate handling of
    - linear motion ($\mathbf{v}$)
    - angular motion ($\boldsymbol{\omega}$)
  - Objects separate



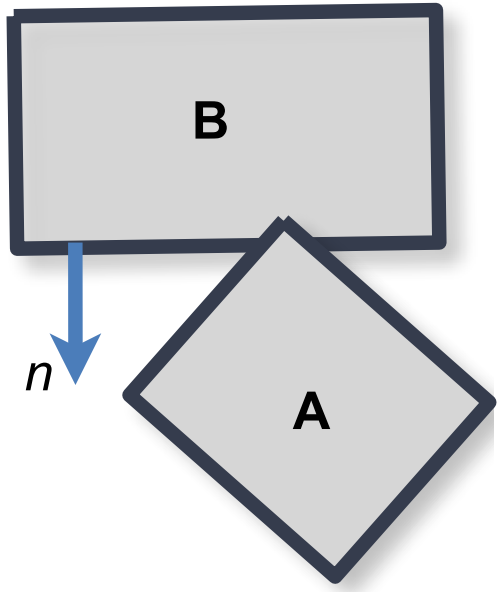
- Simplified, empirical collision model based on c (coefficient of restitution)
  - $c = 1$ Fully elastic, $c = 0$ Fully plastic
  - $c < 1$ Partial kinetic energy transferred into permanently deformed body

$$J = \frac{-(1+c)\mathbf{v}_{rel} \cdot \mathbf{n}}{\frac{1}{M_a} + \frac{1}{M_b} + [(\mathbf{I}_a^{-1}(\mathbf{x}_a \times \mathbf{n})) \times \mathbf{x}_a + (\mathbf{I}_b^{-1}(\mathbf{x}_b \times \mathbf{n})) \times \mathbf{x}_b] \cdot \mathbf{n}}$$

Frictionless Collision Between Two Boxes

- Empirical collision model based on c (coefficient of restitution)

$$v_{rel} = (\mathbf{v}_A - \mathbf{v}_B)$$



- For 2D (The impulse from B to A):

$$J = \frac{-(1+c)\boldsymbol{n} \cdot v_{rel}}{\frac{1}{M_a} + \frac{1}{M_b} + (\mathbf{x}_a \times \mathbf{n})^2/I_a + (\mathbf{x}_b \times \mathbf{n})^2/I_b}$$

- For 3D (The impulse from B to A):

$$J = \frac{-(1+c)\boldsymbol{n} \cdot v_{rel}}{\frac{1}{M_a} + \frac{1}{M_b} + [\mathbf{I}_a^{-1}(\mathbf{x}_a \times \mathbf{n}) \times \mathbf{x}_a + \mathbf{I}_b^{-1}(\mathbf{x}_b \times \mathbf{n}) \times \mathbf{x}_b] \cdot \mathbf{n}}$$

Relative velocity is negative for collision, thus J is always a positive scalar!

- Collision Response:

$$\mathbf{v}_a' = \mathbf{v}_a + J\mathbf{n}/M_a \qquad \mathbf{L}_a' = \mathbf{L}_a + (\mathbf{x}_a \times J\mathbf{n})$$
$$\mathbf{v}_b' = \mathbf{v}_b - J\mathbf{n}/M_b \qquad \mathbf{L}_b' = \mathbf{L}_b - (\mathbf{x}_b \times J\mathbf{n})$$

Frictional Collision Between Two Boxes

- Empirical collision model based on c (coefficient of restitution)

$$\mathbf{v}_{\mathbf{rel}} = (\mathbf{v}_A - \mathbf{v}_B)$$

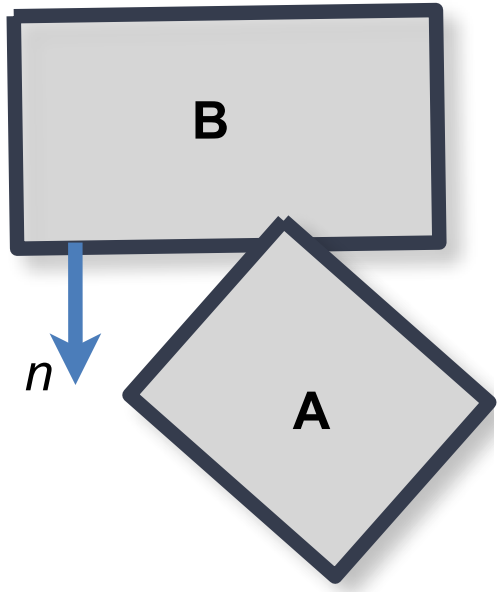
- For 2D (The impulse from B to A):

$$J_n = \frac{-(1+c)\mathbf{v}_{\mathbf{rel}} \cdot \mathbf{n}}{\frac{1}{M_a} + \frac{1}{M_b} + (\mathbf{x}_a \times \mathbf{n})^2/I_a + (\mathbf{x}_b \times \mathbf{n})^2/I_b}$$

Relative velocity is negative for collision, thus J is always a positive scalar!

$$J_t = clamp\left(\frac{-\mathbf{v}_{\mathbf{rel}} \cdot \mathbf{t}}{\frac{1}{M_a}+\frac{1}{M_b}+(\mathbf{x}_a\times\mathbf{n})^2/I_a+(\mathbf{x}_b\times\mathbf{n})^2/I_b}, -uJ_n, uJ_n\right)$$

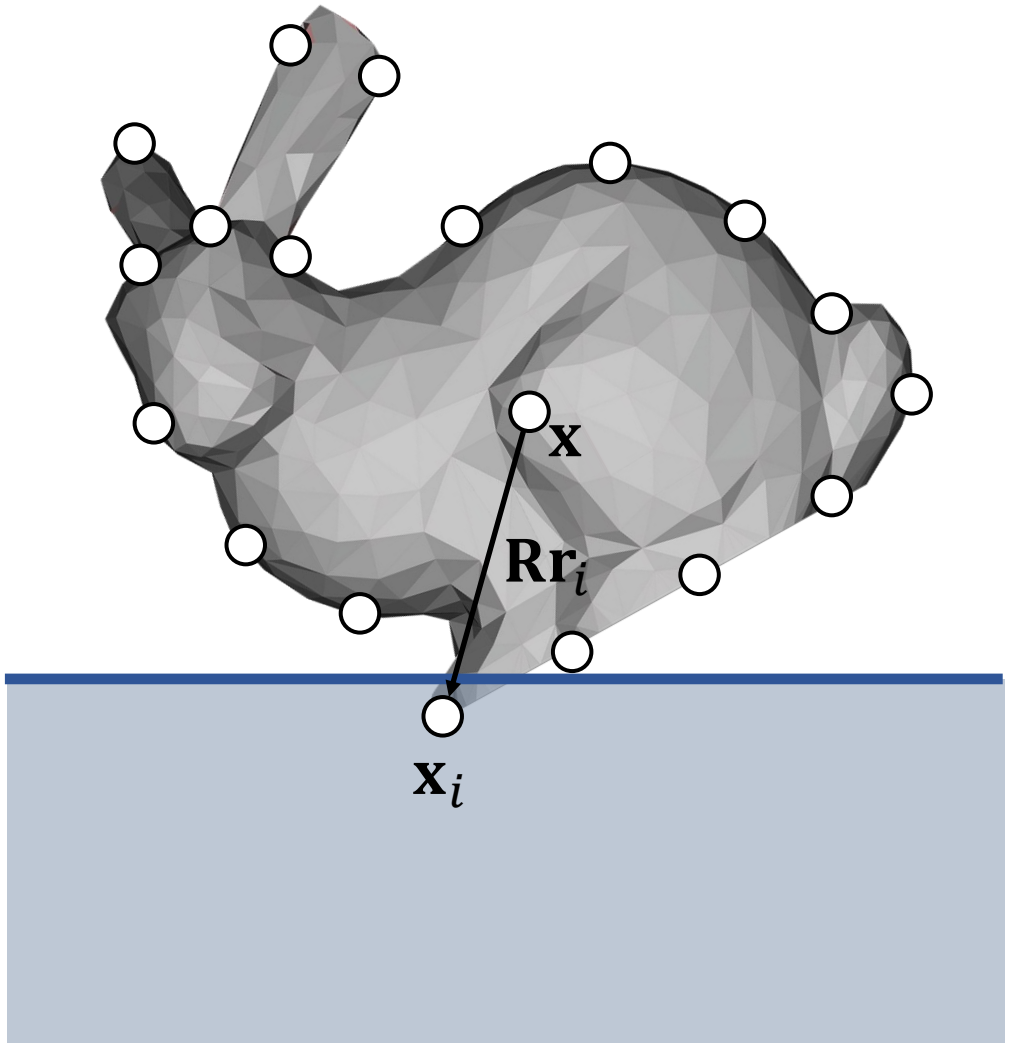
$$\mathbf{J} = J_n\mathbf{n} + J_t\mathbf{t}$$



- Collision Response:

$$\mathbf{v}'_a = \mathbf{v}_a + J\mathbf{n}/M_a \qquad \mathbf{L}'_a = \mathbf{L}_a + (\mathbf{x}_a \times J\mathbf{n})$$
$$\mathbf{v}'_b = \mathbf{v}_b - J\mathbf{n}/M_b \qquad \mathbf{L}'_b = \mathbf{L}_b - (\mathbf{x}_b \times J\mathbf{n})$$

Frictional collision of a rigid rabbit



- Collision Detection:

  When the body is made of many vertices, we can detect its collision by testing each vertex:

$$\mathbf{x}_i \longleftarrow \mathbf{x} + \mathbf{R}\mathbf{r}_i$$

  Not a perfect solution, but acceptable (will come back to this later…)

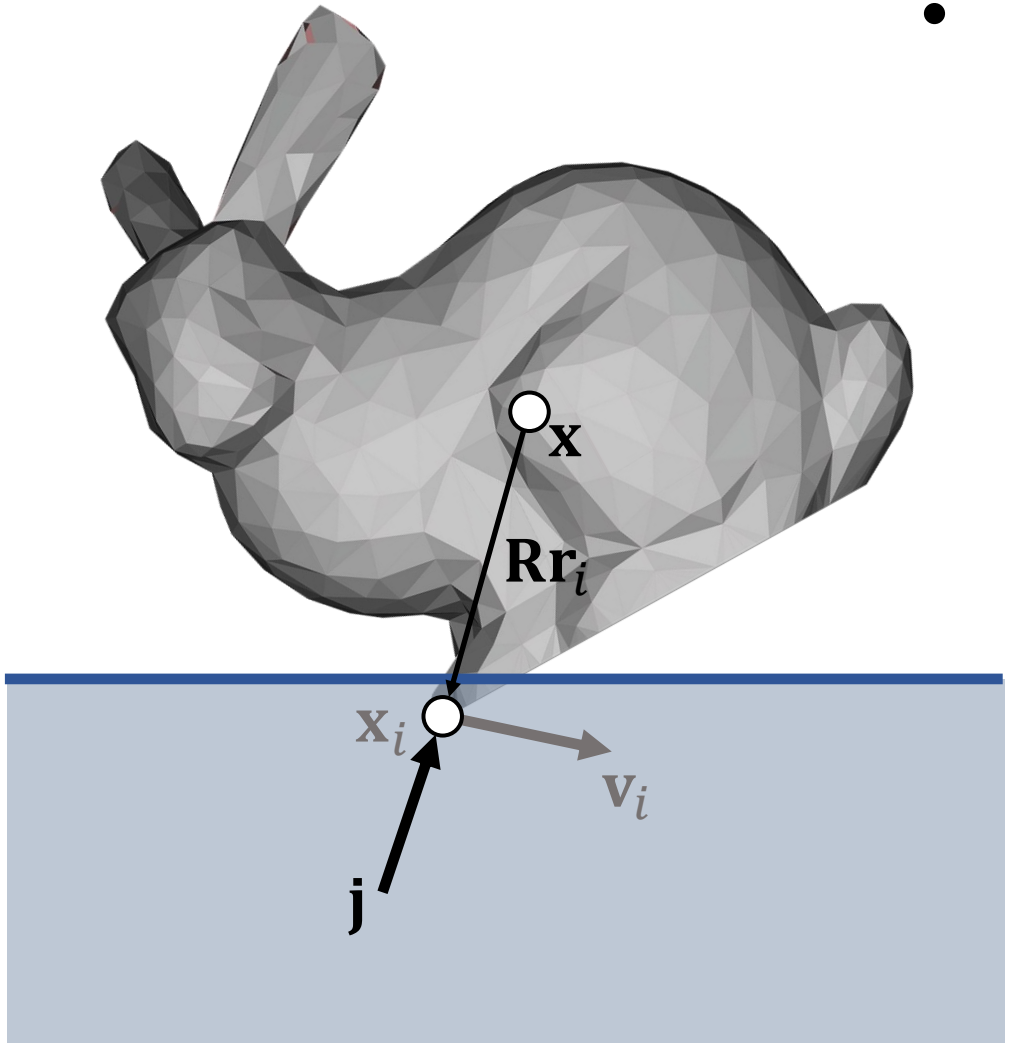
- Classifying Contacts:

$$\mathbf{v}_i \longleftarrow \mathbf{v} + \boldsymbol{\omega} \times \mathbf{R}\mathbf{r}_i$$

$$\mathbf{N} = \nabla\phi(\mathbf{x})$$

$v_{rel} = \boldsymbol{N} \cdot (\mathbf{v}_{\boldsymbol{i}} - \mathbf{v}_{\boldsymbol{B}}) < 0$ **means Collision**

Frictional collision of a rigid rabbit



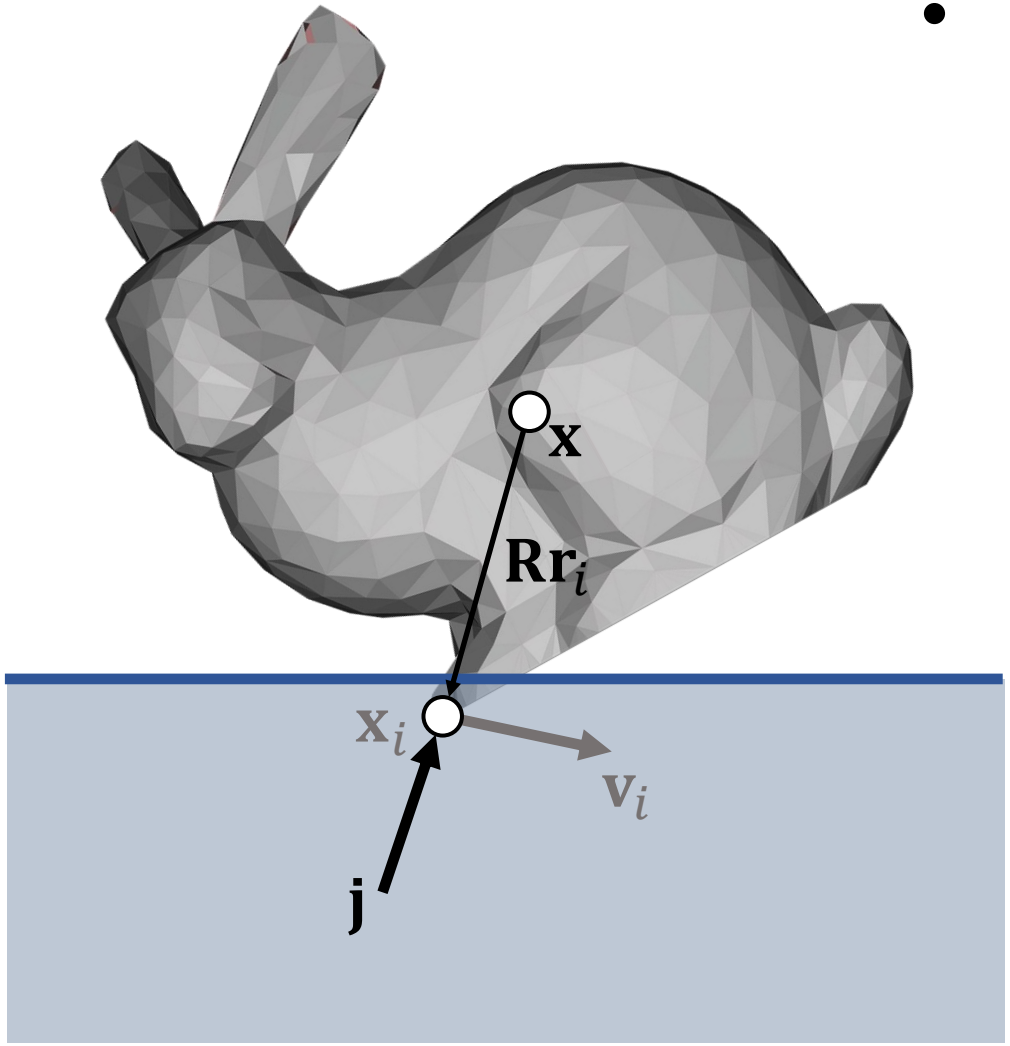
- Collision Response by Impulse

$$\begin{cases} \mathbf{v}^{\text{new}} \longleftarrow \mathbf{v} + \frac{1}{M}\mathbf{j} \\ \boldsymbol{\omega}^{\text{new}} \longleftarrow \boldsymbol{\omega} + \mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j}) \end{cases}$$

If we know how $\mathbf{v}_i$ changes (we know $\mathbf{v}_i^{\text{new}}$), can we calculate the corresponding $\mathbf{j}$ giving such update?

$$\begin{aligned} \mathbf{v}_{\mathbf{N},i} &\longleftarrow (\mathbf{v}_i \cdot \mathbf{N})\mathbf{N} \\ \mathbf{v}_{\mathbf{T},i} &\longleftarrow \mathbf{v}_i - \mathbf{v}_{\mathbf{N},i} \\ a &\longleftarrow \max(1 - \mu_{\mathbf{T}}(1+\mu_{\mathbf{N}})\|\mathbf{v}_{\mathbf{N},i}\| / \|\mathbf{v}_{\mathbf{T},i}\|, 0) \\ \mathbf{v}_{\mathbf{N},i}^{\text{new}} &\longleftarrow -\mu_{\mathbf{N}}\mathbf{v}_{\mathbf{N},i} \\ \mathbf{v}_{\mathbf{T},i}^{\text{new}} &\longleftarrow a\ \mathbf{v}_{\mathbf{T},i} \\ \mathbf{v}_i^{\text{new}} &\longleftarrow \mathbf{v}_{\mathbf{N},i}^{\text{new}} + \mathbf{v}_{\mathbf{T},i}^{\text{new}} \end{aligned}$$

Frictional collision of a rigid rabbit

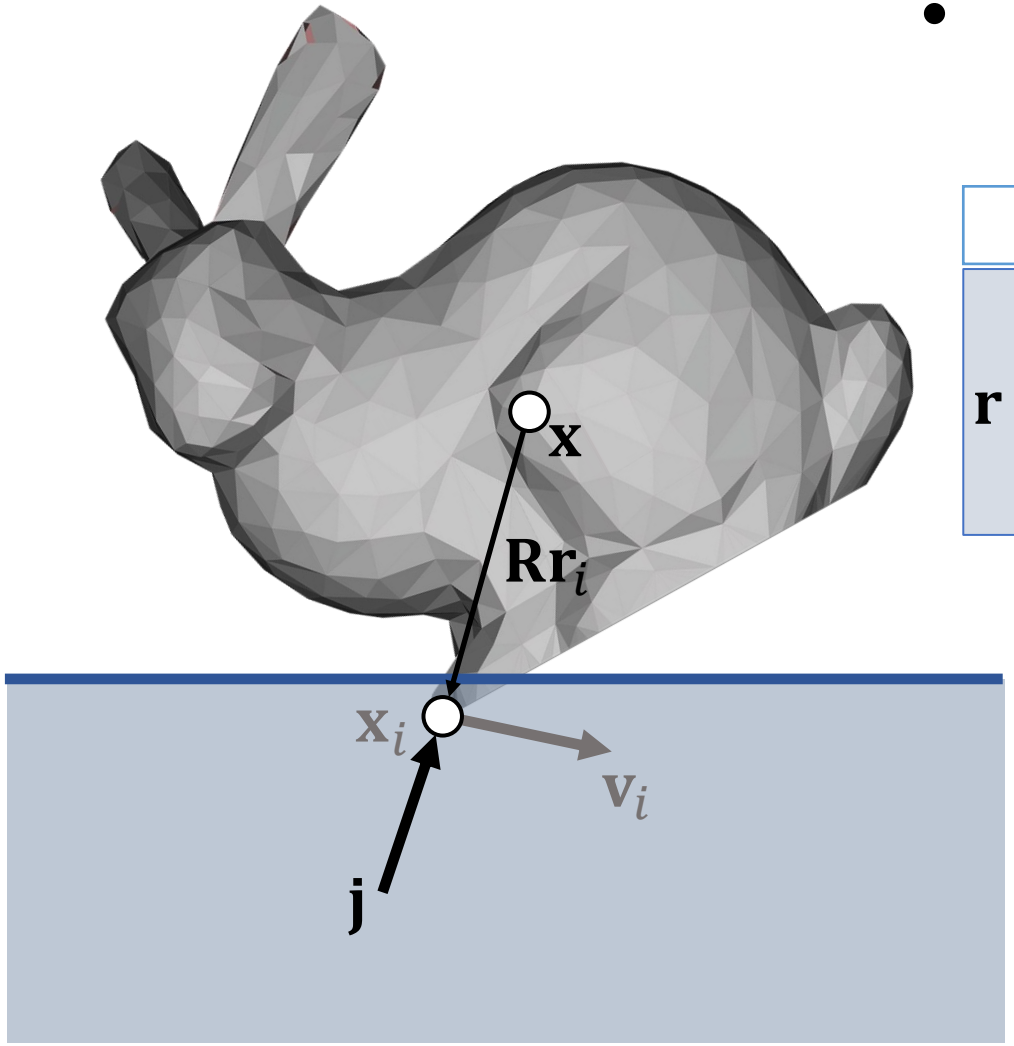


- Collision Response by Impulse

$$\begin{cases} \mathbf{v}^{\text{new}} \longleftarrow \mathbf{v} + \frac{1}{M}\mathbf{j} \\ \boldsymbol{\omega}^{\text{new}} \longleftarrow \boldsymbol{\omega} + \mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j}) \end{cases}$$

$$\begin{aligned} \mathbf{v}_i^{\text{new}} &= \mathbf{v}^{\text{new}} + \boldsymbol{\omega}^{\text{new}} \times \mathbf{R}\mathbf{r}_i \\ &= \mathbf{v} + \frac{1}{M}\mathbf{j} + (\boldsymbol{\omega} + \mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j})) \times \mathbf{R}\mathbf{r}_i \\ &= \mathbf{v}_i + \frac{1}{M}\mathbf{j} + (\mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j})) \times \mathbf{R}\mathbf{r}_i \\ &= \mathbf{v}_i + \frac{1}{M}\mathbf{j} - (\mathbf{R}\mathbf{r}_i) \times (\mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j})) \end{aligned}$$

Frictional collision of a rigid rabbit



- Collision Response by Impulse

$$\mathbf{v}_i^{\text{new}} = \mathbf{v}_i + \frac{1}{M}\mathbf{j} - (\mathbf{R}\mathbf{r}_i) \times (\mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j}))$$

Cross Product ($\mathbf{r} \times$) as a Matrix Product ($\mathbf{r}^*$)

$$\mathbf{r} \times \mathbf{q} = \begin{bmatrix} r_y q_z - r_z q_y \\ r_z q_x - r_x q_z \\ r_x q_y - r_y q_x \end{bmatrix} = \begin{bmatrix} 0 & -r_z & r_y \\ r_z & 0 & -r_x \\ -r_y & r_x & 0 \end{bmatrix} \begin{bmatrix} q_x \\ q_y \\ q_z \end{bmatrix} = \mathbf{r}^* \mathbf{q}$$

$$\mathbf{v}_i^{\text{new}} = \mathbf{v}_i + \frac{1}{M}\mathbf{j} - (\mathbf{R}\mathbf{r}_i)^* \mathbf{I}^{-1} (\mathbf{R}\mathbf{r}_i)^* \mathbf{j}$$

$$\mathbf{v}_i^{\text{new}} - \mathbf{v}_i = \mathbf{K}\mathbf{j}$$

$$\mathbf{K} \longleftarrow \frac{1}{M}\mathbf{1} - (\mathbf{R}\mathbf{r}_i)^* \mathbf{I}^{-1} (\mathbf{R}\mathbf{r}_i)^*$$

Frictional collision of a rigid rabbit

For every vertex $\mathbf{x}_i \longleftarrow \mathbf{x} + \mathbf{R}\mathbf{r}_i$
Test if $\phi(\mathbf{x}_i) < 0$

no → Done

$\mathbf{v}_i \longleftarrow \mathbf{v} + \boldsymbol{\omega} \times \mathbf{R}\mathbf{r}_i$
Is $\mathbf{v}_i \cdot \mathbf{N} < 0$?

no → Done

Compute the wanted $\mathbf{v}_i^{\text{new}}$

$$\mathbf{v}_{\mathbf{N},i} \longleftarrow (\mathbf{v}_i \cdot \mathbf{N})\mathbf{N}$$
$$\mathbf{v}_{\mathbf{T},i} \longleftarrow \mathbf{v}_i - \mathbf{v}_{\mathbf{N},i}$$
$$a \longleftarrow \max(1 - \mu_{\mathbf{T}}(1 + \mu_{\mathbf{N}})\|\mathbf{v}_{\mathbf{N},i}\| / \|\mathbf{v}_{\mathbf{T},i}\|, 0)$$
$$\mathbf{v}_{\mathbf{N},i}^{\text{new}} \longleftarrow -\mu_{\mathbf{N}}\mathbf{v}_{\mathbf{N},i}$$
$$\mathbf{v}_{\mathbf{T},i}^{\text{new}} \longleftarrow a\,\mathbf{v}_{\mathbf{T},i}$$
$$\mathbf{v}_i^{\text{new}} \longleftarrow \mathbf{v}_{\mathbf{N},i}^{\text{new}} + \mathbf{v}_{\mathbf{T},i}^{\text{new}}$$

Compute the impulse $\mathbf{j}$

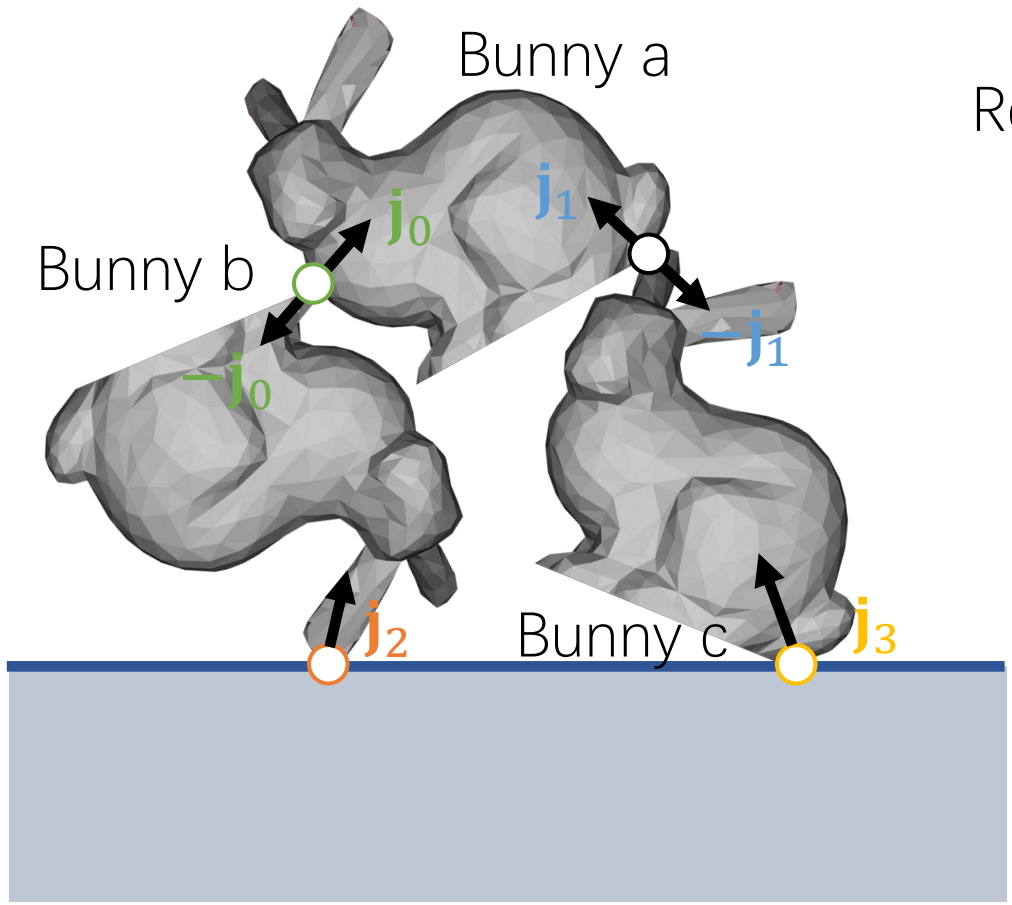
$$\mathbf{K} \longleftarrow \frac{1}{M}\mathbf{1} - (\mathbf{R}\mathbf{r}_i)^*\mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i)^*$$
$$\mathbf{j} \longleftarrow \mathbf{K}^{-1}(\mathbf{v}_i^{\text{new}} - \mathbf{v}_i)$$

Update $\mathbf{v}$ and $\boldsymbol{\omega}$

$$\mathbf{v} \longleftarrow \mathbf{v} + \frac{1}{M}\mathbf{j}$$
$$\boldsymbol{\omega} \longleftarrow \boldsymbol{\omega} + \mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j})$$

→ Done

Multiple Collision by Impulse



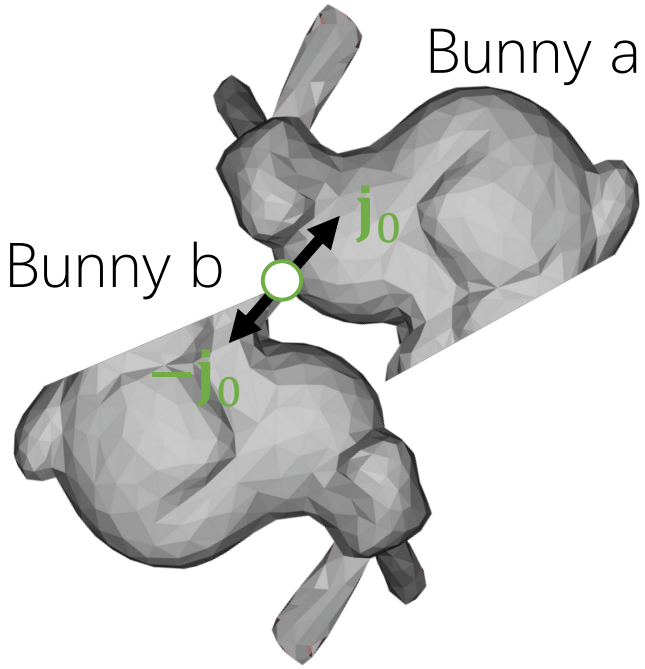
Relative velocity at joints

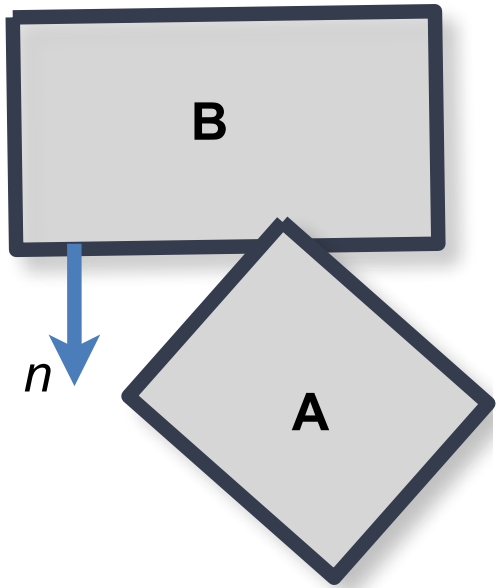
$$
\begin{cases}
\mathbf{v}_0^{\text{new}} - \mathbf{v}_0 = \mathbf{K}_{a00}\mathbf{j}_0 + \mathbf{K}_{a01}\mathbf{j}_1 - (-\mathbf{K}_{b00}\mathbf{j}_0 + \mathbf{K}_{b02}\mathbf{j}_2) \\
\mathbf{v}_1^{\text{new}} - \mathbf{v}_1 = \mathbf{K}_{a10}\mathbf{j}_0 + \mathbf{K}_{a11}\mathbf{j}_1 - (-\mathbf{K}_{c11}\mathbf{j}_0 + \mathbf{K}_{c13}\mathbf{j}_3) \\
\mathbf{v}_2^{\text{new}} - \mathbf{v}_2 = -\mathbf{K}_{b20}\mathbf{j}_0 + \mathbf{K}_{b22}\mathbf{j}_2 \\
\mathbf{v}_3^{\text{new}} - \mathbf{v}_3 = -\mathbf{K}_{c31}\mathbf{j}_1 + \mathbf{K}_{c33}\mathbf{j}_3
\end{cases}
$$

$$
\begin{bmatrix}
\mathbf{K}_{a00} + \mathbf{K}_{b00} & \mathbf{K}_{a01} & -\mathbf{K}_{b02} & \\
\mathbf{K}_{a10} & \mathbf{K}_{a11} + \mathbf{K}_{c11} & & -\mathbf{K}_{c13} \\
-\mathbf{K}_{b20} & & \mathbf{K}_{b22} & \\
 & -\mathbf{K}_{c31} & & \mathbf{K}_{c33}
\end{bmatrix}
\begin{bmatrix} \mathbf{j}_0 \\ \mathbf{j}_1 \\ \mathbf{j}_2 \\ \mathbf{j}_3 \end{bmatrix}
=
\begin{bmatrix} \Delta\mathbf{v}_0 \\ \Delta\mathbf{v}_1 \\ \Delta\mathbf{v}_2 \\ \Delta\mathbf{v}_3 \end{bmatrix}
$$

$\mathbf{K}_{a01}\mathbf{j}_1$ stands for the velocity change of bunny *a* at joint 0, caused by impulse $\mathbf{j}_1$.

Multiple Collision by Impulse





$$\mathbf{v}_0^{\text{new}} - \mathbf{v}_0 = \mathbf{K}_{a00}\mathbf{j}_0 + \mathbf{K}_{b00}\mathbf{j}_0$$

$$\mathbf{K} \leftarrow \frac{1}{M}\mathbf{1} - (\mathbf{R}\mathbf{r}_i)^*\mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i)^*$$

$$\mathbf{J} \leftarrow \mathbf{K}^{-1}(\mathbf{v}_i^{\text{new}} - \mathbf{v}_i)$$

$$\mathbf{x}_a = \mathbf{R}\mathbf{r}_i$$

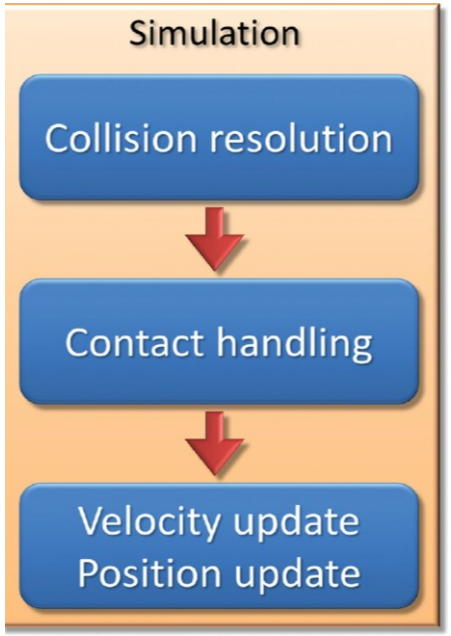
$$J_n = \frac{-(1+c)\mathbf{n}\cdot\mathbf{v}_{\text{rel}}}{\frac{1}{M_a} + \frac{1}{M_b} + [\mathbf{I}_a^{-1}(\mathbf{x}_a \times \mathbf{n}) \times \mathbf{x}_a + \mathbf{I}_b^{-1}(\mathbf{x}_b \times \mathbf{n}) \times \mathbf{x}_b] \cdot \mathbf{n}}$$

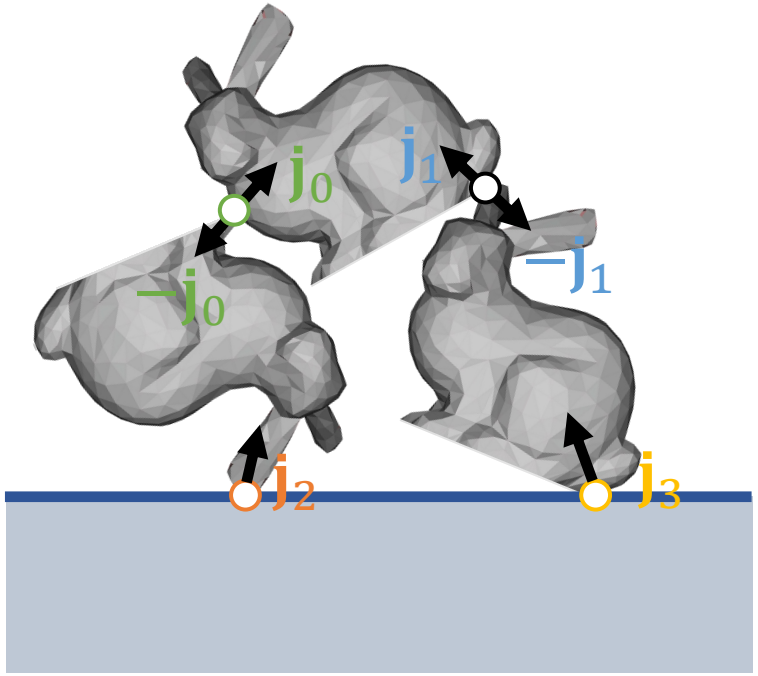
$$\mathbf{J} = J_n\mathbf{n} + J_t\mathbf{t}$$

$\mathbf{K}_{a01}\mathbf{j}_1$ stands for the velocity change of bunny *a* at joint 0, caused by impulse $\mathbf{j}_1$.

Rigid body Collision & Contact

Collision Detection





Collision Detection

Compute the wanted $\mathbf{v}_i^{\text{new}}$

Compute the impulse $\mathbf{j}$

$$\mathbf{K} \longleftarrow \frac{1}{M}\mathbf{1} - (\mathbf{R}\mathbf{r}_i)^*\mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i)^*$$

$$\mathbf{j} \longleftarrow \mathbf{K}^{-1}(\mathbf{v}_i^{\text{new}} - \mathbf{v}_i)$$

Full Update:

Update $\mathbf{v}$ and $\boldsymbol{\omega}$

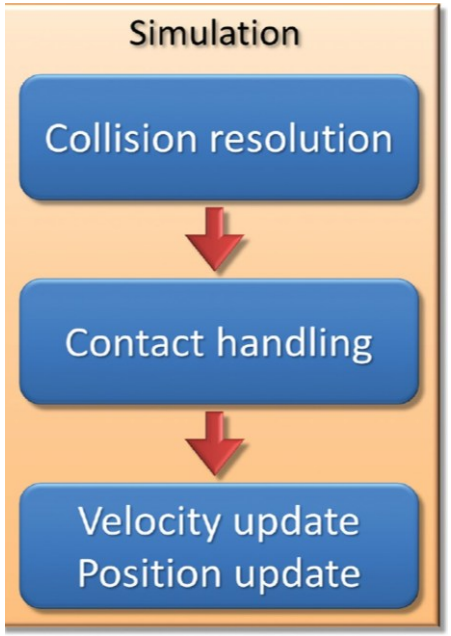
$$\mathbf{v} \longleftarrow \mathbf{v} + \frac{1}{M}\mathbf{j}$$

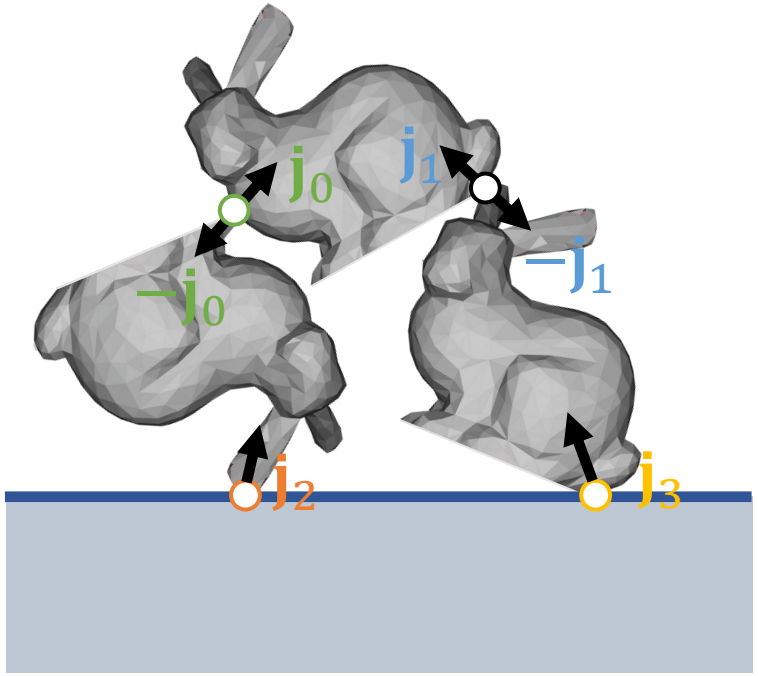
$$\boldsymbol{\omega} \longleftarrow \boldsymbol{\omega} + \mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j})$$

$$\begin{bmatrix} mI_3 & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \dot{v} \\ \dot{\omega} \end{bmatrix} + \begin{bmatrix} 0 \\ \omega \times I\omega \end{bmatrix} = \begin{bmatrix} f \\ \tau \end{bmatrix}$$

Rigid body Collision & Contact

Collision Detection





Collision Detection

Compute the wanted $\mathbf{v}_i^{\text{new}}$

Compute the impulse $\mathbf{j}$

$$\mathbf{K} \longleftarrow \frac{1}{M}\mathbf{1} - (\mathbf{R}\mathbf{r}_i)^*\mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i)^*$$

$$\mathbf{j} \longleftarrow \mathbf{K}^{-1}(\mathbf{v}_i^{\text{new}} - \mathbf{v}_i)$$

Full Update:

$$\mathbf{v} \longleftarrow \mathbf{v} + \frac{1}{M}\mathbf{j} + \Delta t M^{-1}\mathbf{f}$$

$$\boldsymbol{\omega} \longleftarrow \boldsymbol{\omega} + \mathbf{I}^{-1}(\mathbf{R}\mathbf{r}_i \times \mathbf{j}) + \Delta t \mathbf{I}^{-1}(\boldsymbol{\tau} - \boldsymbol{\omega} \times \mathbf{I}\boldsymbol{\omega})$$

$$\mathbf{x}_c \longleftarrow \mathbf{x}_c + \Delta t \mathbf{v}$$

$$\mathbf{q} \longleftarrow \mathbf{q} + \begin{bmatrix} 0 & \frac{\Delta t}{2}\boldsymbol{\omega} \end{bmatrix} \times \mathbf{q}$$

Force-Based & Position-Based & Constraint-Based

Best Practices in Choosing a Method:

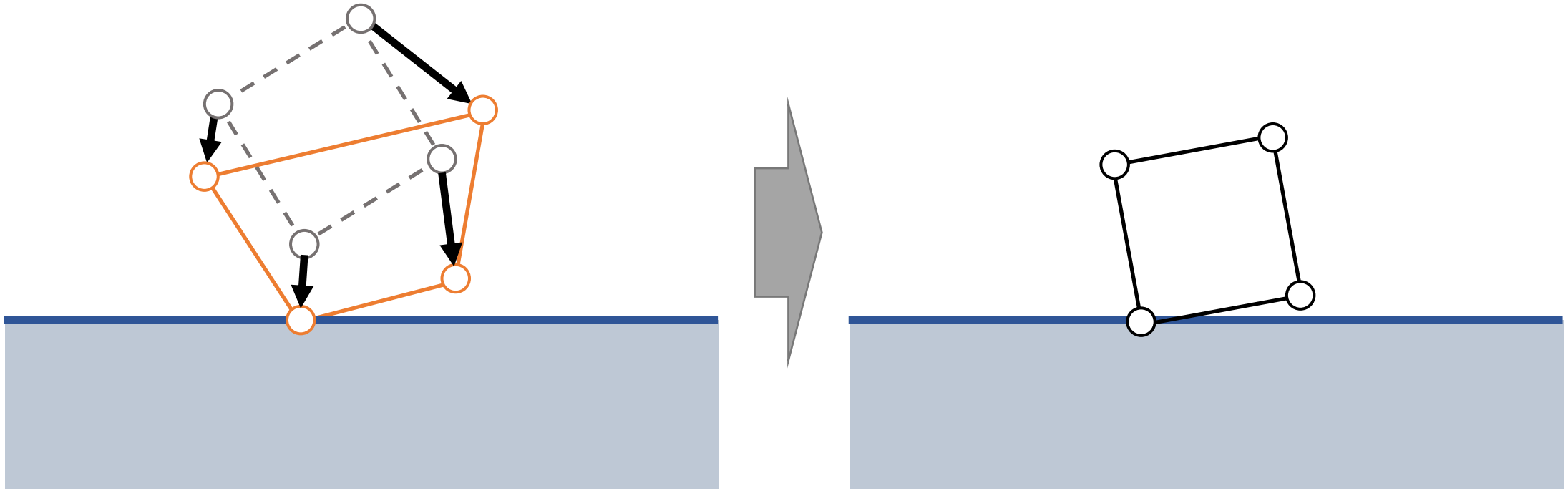
- Games → Position-based
- High-Fidelity Simulation → Impulse + Constraint
- Robotics → Constraint-based

Method	Accuracy	Stability	Performance
Force-Based	Moderate	Needs tuning	Medium
Impulse-Based	High (for collisions)	Unstable for stacking	Medium
Position-Based	Low (not force-based)	**Very Stable**	**High**
Constraint-Based (LCP)	**High** (Global Solve)	Stable	Expensive

Position-Based: Shape Matching

Basic Idea

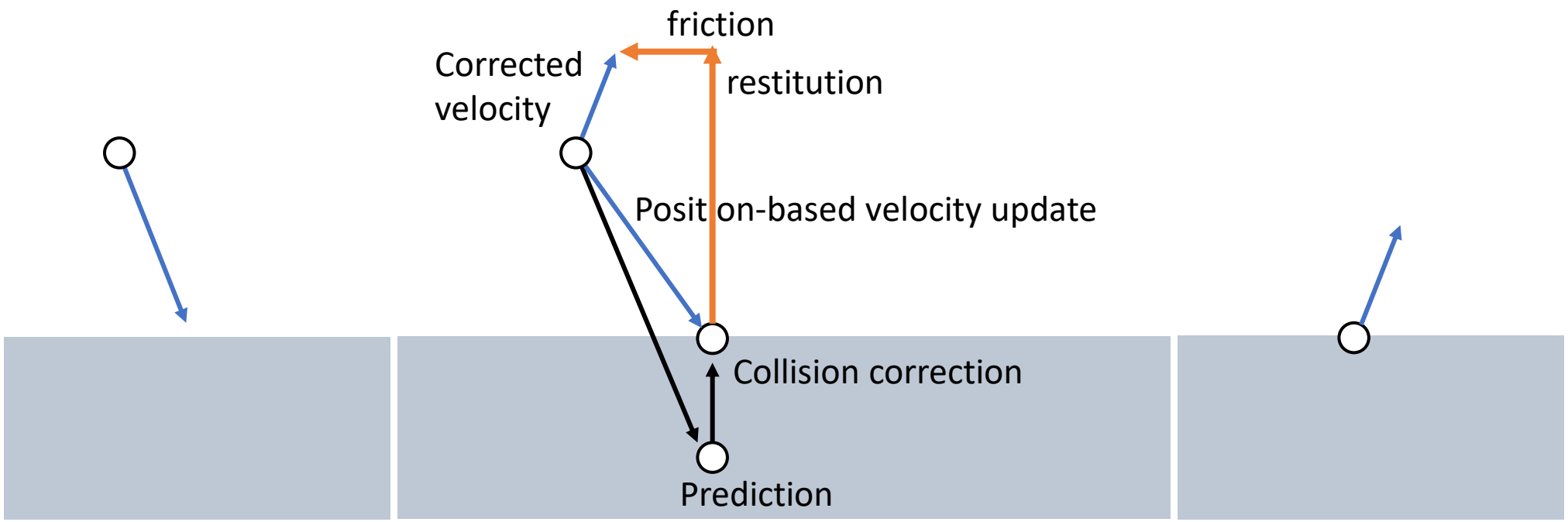
We allow each vertex to have its own velocity, so it can move by itself.



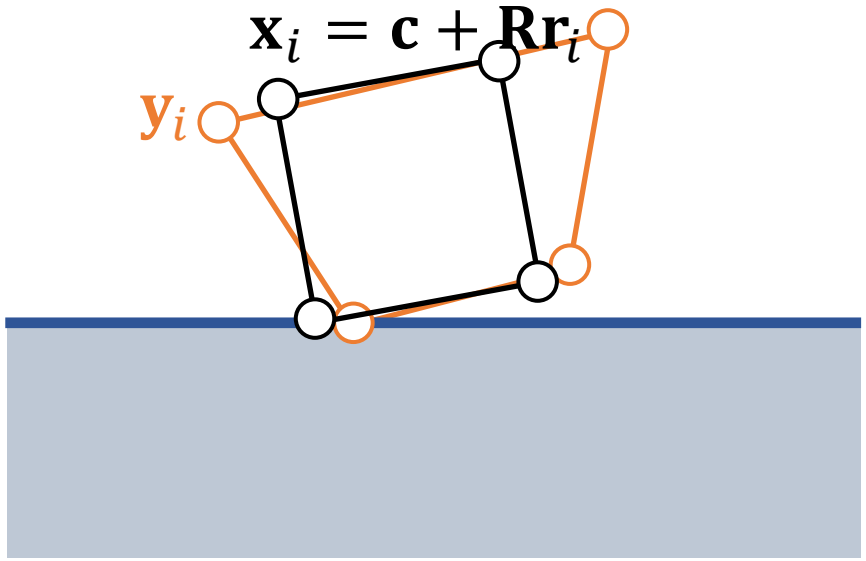
First, move vertices independently by its velocity, with collision and friction being handled.

Second, enforce the rigidity constraint to become a rigid body again.

PBD Step


friction
Corrected velocity
restitution
Position-based velocity update
Collision correction
Prediction

Mathematical Formulation



Now $\mathbf{c}$ and $\mathbf{R}$ are unknowns we want to find out from:

$$\{\mathbf{c}, \mathbf{R}\} = \operatorname{argmin} \sum_i \tfrac{1}{2}\|\mathbf{c} + \mathbf{R}\mathbf{r}_i - \mathbf{y}_i\|^2$$

$$\{\mathbf{c}, \boxed{\mathbf{A}}\} = \operatorname{argmin} \sum_i \boxed{\tfrac{1}{2}\|\mathbf{c} + \mathbf{A}\mathbf{r}_i - \mathbf{y}_i\|^2}$$

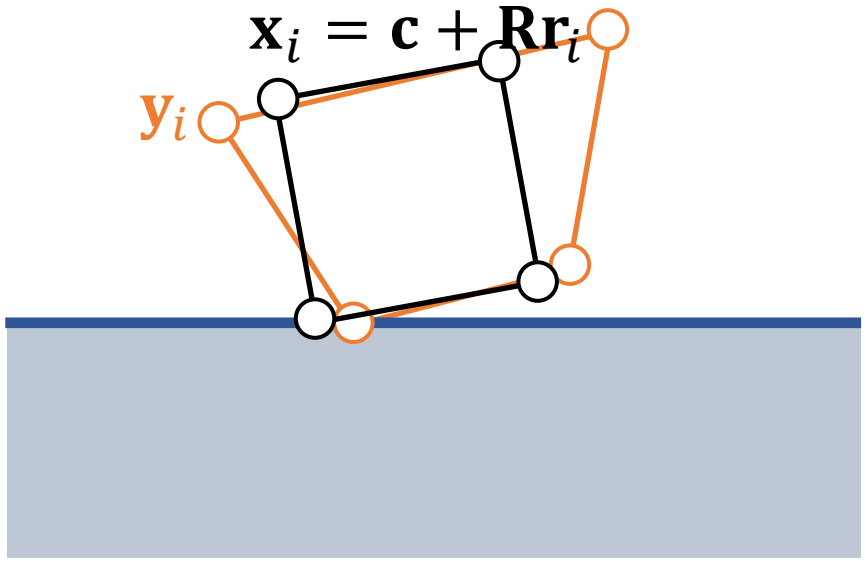
any matrix

The objective E

$$\frac{\partial E}{\partial \mathbf{c}} = \sum_i \mathbf{c} + \mathbf{A}\mathbf{r}_i - \mathbf{y}_i = \sum_i \mathbf{c} - \mathbf{y}_i = \mathbf{0}$$

$$\mathbf{c} = \frac{1}{N}\sum_i \mathbf{y}_i$$

Mathematical Formulation



Next, $$\{\mathbf{c}, \mathbf{A}\} = \operatorname{argmin} \sum_i \frac{1}{2} \|\mathbf{c} + \mathbf{A}\mathbf{r}_i - \mathbf{y}_i\|^2$$

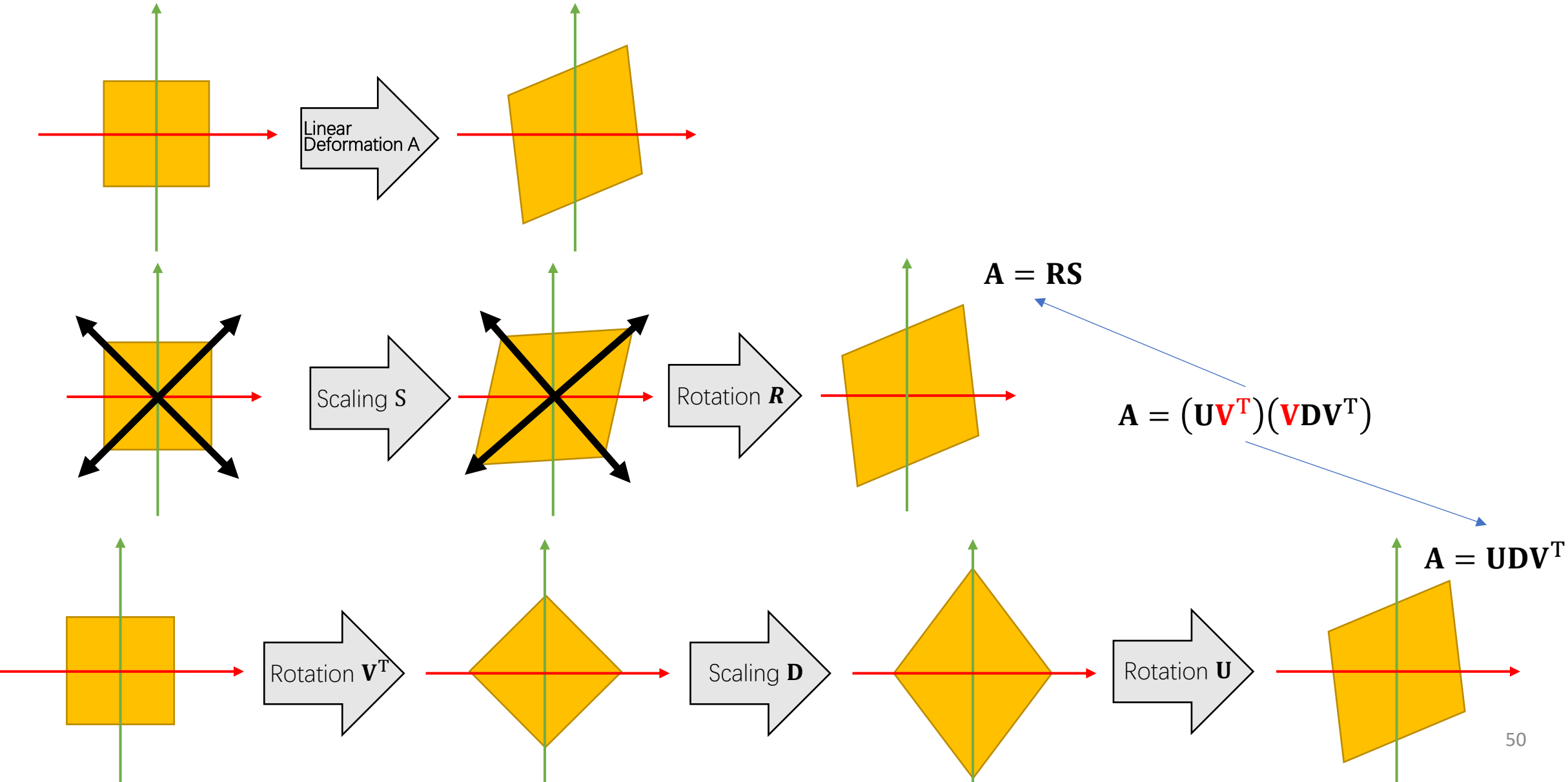
$$\frac{\partial E}{\partial \mathbf{A}} = \sum_i (\mathbf{c} + \mathbf{A}\mathbf{r}_i - \mathbf{y}_i)\mathbf{r}_i^{\mathrm{T}} = \mathbf{0}$$

$$\mathbf{A} = \left(\sum_i (\mathbf{y}_i - \mathbf{c})\ \mathbf{r}_i^{\mathrm{T}}\right)\left(\sum_i \mathbf{r}_i\ \mathbf{r}_i^{\mathrm{T}}\right)^{-1}$$

Polar Decomposition

$$\mathbf{A} = \underbrace{\mathbf{R}}_{\text{rotation}}\ \underbrace{\mathbf{S}}_{\text{deformation}}$$

$\mathbf{A} = \mathbf{U}\mathbf{D}\mathbf{V}^{\mathrm{T}}$ and $\mathbf{A} = \left(\mathbf{U}\mathbf{V}^{\mathrm{T}}\right)\left(\mathbf{V}\mathbf{D}\mathbf{V}^{\mathrm{T}}\right) = \mathbf{R}\mathbf{S}$



Shape Matching

Independent Update

For every vertex

$$\mathbf{f}_i \longleftarrow \mathrm{Force}(\mathbf{x}_i, \mathbf{v}_i)$$

$$\mathbf{v}_i \longleftarrow \mathbf{v}_i + \Delta t m_i^{-1} \mathbf{f}_i$$

$$\mathbf{y}_i \longleftarrow \mathbf{x}_i + \Delta t\, \mathbf{v}_i$$

Handle Collision

Rigidify the vertices

$$\mathbf{c} = \frac{1}{N} \sum_i \mathbf{y}_i$$

$$\mathbf{A} = \left(\sum_i (\mathbf{y}_i - \mathbf{c})\ \mathbf{r}_i^{\mathrm{T}} \right) \left(\sum_i \mathbf{r}_i\ \mathbf{r}_i^{\mathrm{T}} \right)^{-1}$$

$$\mathbf{R} = \mathrm{Polar}(\mathbf{A})$$

Update $\mathbf{v}_i$ and $\mathbf{x}_i$

For every vertex

$$\mathbf{v}_i \longleftarrow (\mathbf{c} + \mathbf{R}\mathbf{r}_i - \mathbf{x}_i)/\Delta t$$

$$\mathbf{x}_i \longleftarrow \mathbf{c} + \mathbf{R}\mathbf{r}_i$$

Done

Physical quantities are attached to each vertex, not to the entire body.

Shape Matching, Some Results

- Nvidia FLEX: Unified Particle Physics for Real-Time Applications

Shape Matching

- Easy to implement and compatible with other nodal systems, i.e., cloth, soft bodies, and even particle fluids.

- Difficult to strictly enforce friction and other goals.

- More suitable when the friction accuracy is unimportant, i.e., buttons on clothes.

After-Class Reading

Muller et al. 2005.
Meshless Deformations Based on Shape Matching.
TOG (SIGGRAPH).

Meshless Deformations Based on Shape Matching

Matthias Müller
NovodeX/AGEIA & ETH Zürich

Bruno Heidelberger
ETH Zürich

Matthias Teschner
University of Freiburg

Markus Gross
ETH Zürich

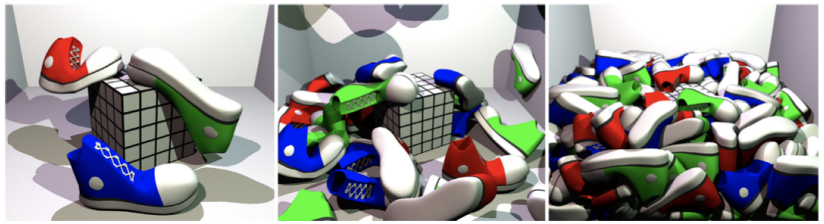


Figure 1: The presented technique is stable under all circumstances and allows to simulate hundreds of deformable objects in real-time.

Abstract

We present a new approach for simulating deformable objects. The underlying model is geometrically motivated. It handles point-based objects and does not need connectivity information. The approach does not require any pre-processing, is simple to compute, and provides unconditionally stable dynamic simulations.

The main idea of our deformable model is to replace energies by geometric constraints and forces by distances of current positions to goal positions. These goal positions are determined via a generalized shape matching of an undeformed rest state with the current deformed state of the point cloud. Since points are always drawn towards well-defined locations, the overshooting problem of explicit integration schemes is eliminated. The versatility of the approach in terms of object representations that can be handled, the efficiency in terms of memory and computational complexity, and the unconditional stability of the dynamic simulation make the approach particularly interesting for games.

CR Categories: I.3.5 [Computer Graphics]: Computational Geometry and Object Modeling—Physically Based Modeling; I.3.7 [Computer Graphics]: Three-Dimensional Graphics and Realism—Animation and Virtual Reality

Keywords: deformable modeling, geometric deformation, shape matching, real-time simulation

1 Introduction

Since Terzopoulos' pioneering work on simulating deformable objects in the context of computer graphics [Terzopoulos et al. 1987], many deformable models have been proposed. In general, these approaches focus on an accurate material representation, on stability aspects of the dynamic simulation and on versatility in terms of advanced object characteristics that can be handled, e. g. plastic deformation or fracturing.

Despite the long history of deformable modeling in computer graphics, research results have rarely been applied in computer games. Nowadays, deformable cloth models with simple geometries can be found in a few games, but in general, games are dominated by rigid bodies. Although rigid bodies can be linked with joints to represent articulated structures, there exist no practical solution which allows to simulate elastically deformable three-dimensional objects in a stable and efficient way. There are several reasons that prevent current deformable models from being used in interactive applications.

Efficiency. Existing deformable models based on complex material laws in conjunction with stable, implicit integration schemes are computationally expensive. Such approaches do not allow for interactive simulations of objects with a reasonable geometrical complexity. Further, these approaches might require a specific object representation and the algorithms can be hard to implement and debug. In contrast, interactive applications such as games constitute hard constraints on the computational efficiency of a deformable modeling approach. The approach is only allowed to use a small fraction of the available computing resources. Further, specific volumetric representations of deformable objects are often not available since the geometries are typically represented by surfaces only.

Stability. In interactive applications, the simulation of deformable objects needs to remain stable under all circumstances. While sophisticated approaches allow for stable numerical integration of velocities and positions, additional error sources such as degenerated geometries, physically incorrect states, or problematic situations with large object interpenetrations are not addressed by many approaches. A first contribution to this research area has been presented in [Irving et al. 2004], where large deformations and the inversion of elements in FE approaches can be handled in a robust way. However, this approach is not intended to be used in interactive applications.

After-Class Reading

Muller et al. 2022. *Physically Based Shape Matching.* SCA 2022

ACM SIGGRAPH / Eurographics Symposium on Computer Animation 2022
D. L. Michels and S. Pirk
(Guest Editors)

Volume 41 (2022), Number 8

Physically Based Shape Matching

Matthias Müller Miles Macklin Nuttapong Chentanez Stefan Jeschke

NVIDIA

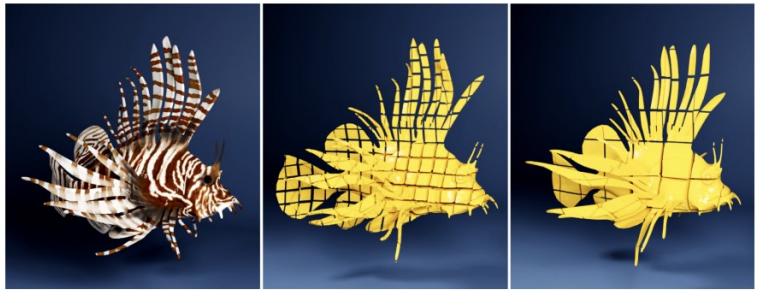


Figure 1: *Our method allows the direct simulation of a cut-cell mesh comprised of arbitrarily shaped elements. The simulation mesh matches the input surface mesh perfectly. Even at low resolution, the lion fish deforms in a plausible way.*

Abstract

The shape matching method is a popular approach to simulate deformable objects in interactive applications due to its stability and simplicity. An important feature is that there is no need for a mesh since the method works on arbitrary local groups within a set of particles. A major drawback of shape matching is the fact that it is geometrically motivated and not derived from physical principles which makes calibration difficult. The fact that the method does not conserve volume can yield visual artifacts, e.g. when a tire is compressed but does not bulge.

In this paper we present a new meshless simulation method that is related to shape matching but derived from continuous constitutive models. Volume conservation and stiffness can be specified with physical parameters. Further, if the elements of a tetrahedral mesh are used as groups, our method perfectly reproduces FEM based simulations.

Keywords: finite element method, physically-based animation, elasticity, real-time physics

CCS Concepts
• **Computing methodologies → Physical simulation;**

1. Introduction

Simulating deformable objects has a long history in computer graphics. Applications are flesh simulations on characters or rubbery objects like tires. A large variety of methods have been proposed in this field. Many of those methods are mesh-based meaning the simulation is performed on a volumetric mesh enclosing an object. Since creating such a mesh is a difficult problem, mesh generation is a large field in itself.

Meshless methods have been introduced to avoid this problem. Here, objects are typically sampled by particles. The particles are held together by constraints or forces acting on local particle groups. Other than not requiring a mesh, they have a number of additional advantages. For instance, topological changes can be handled by simply including or excluding particles from groups. When large plastic deformations occur, objects can be re-sampled easily and the groups re-created based on particle distances.

Mass spring networks are a simple example. Here, the interac-

© 2022 The Author(s)
Computer Graphics Forum © 2022 The Eurographics Association and John Wiley & Sons Ltd. Published by John Wiley & Sons Ltd.